



THE COMPLETE LAB MANUAL

Secure Network Engineering Lab

Build it from A to Z: networking, identity, firewalls, proxies, VPN, IDS/IPS, hardening, cloud, PKI, containers, Kubernetes, IaC, CI/CD, detection engineering, and a purple-team capstone.

```
$ ./deploy -modules all -learn-by-doing
```

How to use this manual

This is not a book to read once. It is a **lab you build**. Every module follows the same rhythm so you always know what to do:

- **Theory** boxes explain *why* something works, kept practical, not academic.
- **Objective** boxes tell you exactly what you will have working by the end of a task.
- Dark code blocks are commands you type; light bordered blocks are configuration files.
- **Verify** boxes give you a concrete test so you *know* it worked.
- **Red Team** boxes attack what you just built so you learn defence by experiencing offence.

Golden rule of this lab: after every defensive control you build, immediately try to break it. The attack/defend loop is what turns you from someone who follows tutorials into an engineer who understands systems.

WATCH OUT

Everything here runs inside **your own isolated lab**. Only scan, brute-force, or attack machines you own and that live on your lab network. Pointing these tools at systems you do not own is illegal in most countries. Keep the lab network isolated from the public internet for the offensive modules.

Contents

How to use this manual	i
0 The Lab Blueprint	1
0.1 What you are building	1
0.1.1 Hardware & host software	1
0.1.2 The virtual machines	1
0.2 Network topology	2
0.3 The master IP addressing plan	3
1 Central Identity & Access Management	4
1.1 Stand up the directory	4
1.1.1 Option A – Windows Server Active Directory	4
1.1.2 Option B – FreeIPA (recommended for this all-Linux lab)	5
1.2 Join your hosts & kill local-only auth	5
1.3 RBAC: grant rights to groups, not people	6
1.4 Red Team – why the directory is the crown jewel	6
2 Networking Foundations (Practical)	7
2.1 The OSI model, the practical version	7
2.2 IP addressing & subnetting	7
2.3 TCP vs UDP and ports	8
2.4 Routing	8
2.5 NAT and PAT	9
2.6 DNS	9
2.7 DHCP	9
2.8 VLANs	10
2.9 HTTP and HTTPS	10
2.10 Forward proxy vs reverse proxy	10
2.11 Firewall logic: stateful vs stateless	11
3 The Firewall: Routing, NAT & Rules	12
3.1 Bring up WAN and LAN	12
3.1.1 Assign interfaces (pfSense)	12
3.1.2 The same idea on a plain Linux router (iptables)	12
3.2 Stateful baseline: the default-deny posture	13
3.3 Block specific ports	13
3.4 Allow only certain source IPs	13
3.5 Permit one machine, deny another	14
3.6 Allow only HTTP (a tightly scoped policy)	14
3.7 Configure NAT and port forwarding	14
3.8 Block SSH from the internet	15
3.9 DMZ-style segmentation	15
3.10 Make iptables rules survive a reboot	16

4	Proxies: Filtering, Logging & Control	17
4.1	Forward proxy with Squid	17
4.1.1	Point a client at the proxy	17
4.2	URL / domain filtering: restrict websites	17
4.3	Logging requests (your audit trail)	18
4.4	Force traffic through the proxy (transparent proxy)	19
4.5	Reverse proxy with Nginx (protect the DMZ server)	19
4.6	TLS interception (SSL Bump)	20
4.6.1	Generate the proxy CA and configure bumping	20
4.6.2	Trust the CA on the client	21
4.7	Authenticate the proxy against the directory	21
5	VPN: Secure Remote Access	23
5.1	WireGuard (modern, recommended)	23
5.1.1	Server side (the firewall / a Linux gateway)	23
5.1.2	Client side	24
5.2	OpenVPN (the classic)	24
6	IDS/IPS & Attack Simulation	25
6.1	Install Suricata (IDS mode first)	25
6.2	Attack 1 – Port scanning (reconnaissance)	26
6.3	Attack 2 – SSH brute force	26
6.4	Attack 3 – Web scanning	26
6.5	Turn Suricata into an IPS (block, not just alert)	27
6.6	On-premises WAF (ModSecurity + OWASP CRS)	27
6.6.1	Tune before you block (the real WAF lesson)	28
7	Host Hardening	29
7.1	SSH hardening	29
7.2	Host firewall with ufw (and the iptables underneath)	29
7.3	Fail2Ban: auto-ban attackers	30
7.4	Patch management	30
7.5	Least privilege & secure permissions	31
7.6	Fleet management with Ansible	31
7.7	Vulnerability scanning (find what to patch)	32
7.7.1	Prioritise: you cannot patch everything	33
7.8	Multi-factor authentication on SSH and VPN	34
8	Integrated Detection: Attack to Dashboard	35
8.1	Build the dashboard (SIEM)	35
8.2	Centralise the logs	35
8.3	Chain 1 – SSH brute force, blocked by Fail2Ban	36
8.4	Chain 2 – Kali attack → firewall log → IDS alert → dashboard	36
8.5	Detection engineering: write your own rules	37
8.5.1	Decode the log, then write the rule	37
8.5.2	Make detections portable: Sigma	38
9	Cloud Security (AWS)	39
9.1	VPC and subnets	39
9.2	Security Groups (stateful) vs NACLs (stateless)	39
9.3	NAT Gateway	40
9.4	Bastion host	40
9.5	Load balancer + WAF	40

9.6	IAM least privilege	41
9.7	Logging & monitoring: CloudTrail + CloudWatch	41
10	PKI & Certificate Authority	43
10.1	Build a two-tier internal CA	43
10.2	Mutual TLS (mTLS): both sides prove identity	44
10.3	Automated issuance & revocation	45
11	Linux Internals & nftables	46
11.1	nftables: the modern packet filter	46
11.2	Namespaces & cgroups: how isolation works	47
11.3	Capabilities: break up root	47
11.4	seccomp & Mandatory Access Control	48
11.5	PAM, systemd & journald: the auth and service plumbing	48
12	Windows Enterprise Security & Detection	50
12.1	Windows logging: the events that matter	50
12.2	Sysmon: the defender's microscope	51
12.3	Ship Windows logs to your SIEM	51
12.4	Detect the AD attacks you already ran	51
12.5	Windows hardening enterprises actually use	52
13	Container & Kubernetes Security	53
13.1	Docker: harden the image and the runtime	53
13.2	Kubernetes: a hardened cluster	54
13.2.1	RBAC: least privilege for the cluster	54
13.2.2	Network policies: default-deny inside the cluster	55
13.2.3	Pod Security, secrets, and admission control	55
13.2.4	Runtime detection with Falco	55
14	Infrastructure as Code: Terraform, Ansible & Secrets	57
14.1	Provision with Terraform	57
14.2	Scan the code before it ships	58
14.3	State files are secret – treat them so	58
14.4	Secrets management with Vault	58
15	CI/CD & Supply-Chain Security	60
15.1	The security gates in a pipeline	60
15.2	Build a gated pipeline	60
15.3	Protect the supply chain	61
15.4	Harden the pipeline itself	61
16	Advanced AWS Security	63
16.1	Threat detection: GuardDuty & Security Hub	63
16.2	Encryption with KMS	63
16.3	S3 hardening	64
16.4	Secrets Manager	64
16.5	IAM at depth	64
17	Monitoring & Observability	66
17.1	Stand up the metrics stack	66
17.2	Alert on security signals, not just outages	66
17.3	Traces and the observability/SIEM divide	67

18 Detection Engineering at Depth	68
18.1 The detection lifecycle	68
18.2 Map coverage to MITRE ATT&CK	68
18.3 Validate detections by emulating attacks	69
18.4 Threat hunting: look for what rules miss	69
19 Incident Response & Digital Forensics	70
19.1 Triage: is this real, and how bad?	70
19.2 Containment without destroying evidence	70
19.3 Evidence acquisition & integrity	71
19.4 Analysis: build the timeline	71
19.5 Eradication, recovery, and the post-incident review	72
20 Capstone: Purple-Team Exercise	73
20.1 Threat-model the lab first	73
20.2 The kill chain: attack the whole lab	73
20.3 Verify the defence, end to end	74
20.4 Where to take it next	74
21 Skills Checklist & Study Path	75
21.1 Mastery checklist	75
21.2 Suggested order & pacing	76
A Command Cheat-Sheet	78
A.1 Networking & diagnostics	78
A.2 Firewall (iptables / ufw)	78
A.3 Services & logs	78
B Troubleshooting Playbook	79
C Where to Keep Learning	80

MODULE 0

The Lab Blueprint

Before a single rule is written, you need a target architecture. A security lab is just a small enterprise network you fully control. This module defines the hardware, the virtual machines, the network segments, and the IP plan that every later module assumes.

0.1 What you are building

◎ OBJECTIVE

A segmented virtual network with a firewall at its core, separating a trusted LAN, an internet-facing DMZ, and a management network, plus a dedicated attacker box. By the end of the manual every device, rule, proxy, sensor, and log pipeline shown below is live and tested.

0.1.1 Hardware & host software

You can run the whole lab on one reasonably modern machine. Minimum comfortable spec: a 4-core CPU with virtualisation enabled (VT-x/AMD-V), 16 GB RAM, and 100 GB free disk. 32 GB RAM makes life smoother once IDS and a SIEM are running.

Choice	Use it if	Notes
VirtualBox / VMware Workstation	You have one lap-top/PC	Easiest start. Create internal networks to isolate segments.
Proxmox VE (bare metal)	You have a spare PC/mini-PC	Closest to a real datacentre; VLAN-aware bridges; learn a real hypervisor.

0.1.2 The virtual machines

VM	Role	Image
fw-pfsense	Firewall / router / NAT	pfSense CE or OPNsense
dc-ipa	Domain controller (identity)	Windows Server AD or FreeIPA
proxy-squid	Forward + reverse proxy	Ubuntu Server LTS
web-dmz	DMZ web server (target)	Ubuntu Server + Nginx
lan-client	Trusted workstation	Ubuntu Desktop or Debian
ids-sensor	IDS/IPS + SIEM dashboard	Ubuntu Server (Suricata + Wazuh/ELK)
mgmt-ansible	Config mgmt + vuln scanner	Ubuntu Server (Ansible + OpenVAS)
attacker-kali	Red-team box	Kali Linux

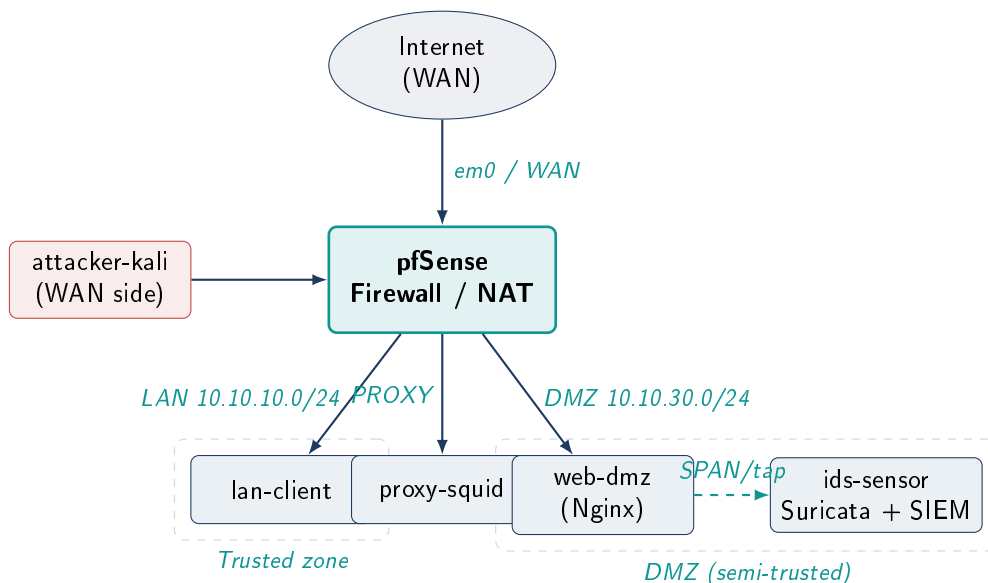
Advanced track (add when you reach these modules):

VM	Role	Image
docker-host	Container security	Ubuntu Server + Docker
k8s-node	Kubernetes security	Ubuntu Server + k3s/minikube
ci-runner	CI/CD + supply chain	Ubuntu Server + a CI runner
win-endpoint	Windows detection (Sysmon)	Windows 10/11 + Sysmon
bastion	Jump host (sole SSH ingress)	Ubuntu Server, hardened
monitoring	Observability stack	Ubuntu (Prometheus + Grafana)
soc-workstation	Detection & IR analyst box	Ubuntu Desktop + forensic tools
vuln-app	Deliberately vulnerable target	DVWA / OWASP Juice Bench

Enterprise reference architecture. Beyond the simple topology below, a realistic estate groups machines into *trust zones* and forces traffic through choke points. The mapping you are building toward: an *Edge/DMZ* zone (only the reverse proxy + WAF face the internet), an *Internal/User* zone (workstations, the directory), a *Server/App* zone (containers, databases – no inbound from the internet), a *Security/SOC* zone (SIEM, monitoring, IR workstation – it receives logs but exposes nothing), and an out-of-band *Management* zone reached only through the **bastion**. Trust decreases as you move toward the edge; the firewall enforces that no lower-trust zone can initiate into a higher-trust one. Logs flow *one way* into the Security zone; admin access flows *one way* out through the bastion. That asymmetry is the whole game.

0.2 Network topology

The firewall has four legs. Each leg is a separate virtual network (an "internal network" in VirtualBox, or a VLAN-tagged bridge in Proxmox) so traffic between segments *must* pass through the firewall, where you can inspect and filter it.



THEORY

Why segment at all? A flat network means one compromised laptop can reach every other device. Segments + a firewall between them enforce the principle that a public web server (which *will* get attacked) cannot turn around and reach your trusted workstations. This is the same logic as a DMZ in a real company: the riskiest machines live in their own zone with tightly controlled paths in and out.

0.3 The master IP addressing plan

Write this down once and reuse it everywhere. Consistent addressing is what keeps a lab understandable months later.

Segment	Network	Firewall IP	Hosts
WAN	DHCP from host / 192.168.1.0/24	dynamic	attacker-kali (external)
LAN (trusted)	10.10.10.0/24	10.10.10.1	dc .5, lan-client .10, proxy .20
DMZ	10.10.30.0/24	10.10.30.1	web-dmz .10, ids-sensor .50
MGMT	10.10.99.0/24	10.10.99.1	mgmt-ansible .10 (Ansible + scanner)

Subnetting refresher you will actually use here. A /24 mask (255.255.255.0) gives 256 addresses, 254 usable for hosts, with the network address .0 and broadcast .255 reserved. The firewall always takes .1 (the default gateway). If you ever need smaller segments, a /28 (255.255.255.240) gives 16 addresses / 14 hosts – handy for a tiny DMZ. Module 2 drills this properly.

MODULE 1

Central Identity & Access Management

So far every host has its own local users – fine for three machines, unworkable for thirty. Real organisations centralise identity: one directory authenticates every user on every system, with group-based access control. This module stands up that directory, joins your Linux hosts to it, and disables local-only logins. It also opens the door to the single most important attack surface in modern enterprises.

THEORY

Why this is the layer attackers want most. A directory (Active Directory or FreeIPA) issues **Kerberos** tickets: you authenticate once, and a time-limited ticket then proves your identity to every service – no passwords flying around. **LDAP** is the database underneath (users, groups, hosts). **RBAC** (role-based access control) means you grant rights to *groups*, and membership decides access. Compromising the directory means compromising *everything* authenticated by it, which is exactly why directory attacks (Kerberoasting, AS-REP roasting, ticket forgery) dominate real red-team engagements. You cannot practise defending them without a directory to defend.

1.1 Stand up the directory

🎯 OBJECTIVE

A domain controller on the trusted LAN at 10.10.10.5 serving Kerberos + LDAP for the realm LAB.LOCAL, with at least one domain user and one access group.

You have two realistic choices. Pick based on your goal:

Option	Choose if	Notes
Windows Server AD	You want job-relevant skills	AD is what you will attack/defend in the field; richest tooling.
FreeIPA (Linux)	You want lighter & Linux-native	Same Kerberos + LDAP + RBAC concepts; one command to install.

1.1.1 Option A – Windows Server Active Directory

On a Windows Server VM, promote it to a domain controller (PowerShell, run as admin):

```
Install-WindowsFeature AD-Domain-Services -IncludeManagementTools
Install-ADDSForest -DomainName "lab.local" -InstallDns
# after reboot, create a user and a group:
New-ADUser -Name "jdoe" -SamAccountName jdoe -Enabled $true -AccountPassword (Read-Host -AsSecureString)
New-ADGroup -Name "linux-admins" -GroupScope Global
Add-ADGroupMember -Identity "linux-admins" -Members jdoe
```

1.1.2 Option B – FreeIPA (recommended for this all-Linux lab)

```
# on the DC VM (give it a static IP 10.10.10.5 and a proper hostname first)
sudo hostnamectl set-hostname dc.lab.local
sudo apt install freeipa-server -y          # or: dnf install ipa-server (RHEL family)
sudo ipa-server-install --domain=lab.local --realm=LAB.LOCAL --setup-dns
# authenticate as admin, then create a user and an access group:
kinit admin
ipa user-add jdoe --first=John --last=Doe
ipa group-add linux-admins --desc="Linux administrators"
ipa group-add-member linux-admins --users=jdoe
```

WATCH OUT

Kerberos lives and dies by the clock. Tickets are time-stamped, and a skew greater than ~5 minutes between a client and the DC breaks authentication completely – with confusing errors. Run **chrony**/NTP on the DC and point every host at it *before* you join anything. This single gotcha causes most "it just won't authenticate" pain.

1.2 Join your hosts & kill local-only auth

🎯 OBJECTIVE

lan-client and **proxy-squid** authenticate users against the directory; logging in with a domain account works on both.

```
# point the host's DNS + time at the DC, then join the realm
sudo apt install realmd sssd sssd-tools adcli krb5-user chrony -y
sudo realm join lab.local -U admin          # AD; for FreeIPA use: ipa-client-install
# allow domain logins and auto-create home dirs on first login
sudo realm permit --all
echo 'session optional pam_mkhomedir.so' | sudo tee -a /etc/pam.d/common-session
```

VERIFY IT WORKS

The domain user resolves and can log in, and gets a Kerberos ticket:

```
id jdoe@lab.local          # shows domain UID + group memberships
su - jdoe@lab.local        # log in as the domain user
kinit jdoe && klist         # request a ticket; klist shows it cached
ssh jdoe@lab.local@10.10.10.20 # SSH to the proxy using domain credentials
```

WATCH OUT

Keep a break-glass account. Before you tighten anything, confirm one *local* admin account still works on each host. If the DC is ever down (or you misconfigure SSSD) and you have *only* domain auth, you are locked out of every machine at once. Every serious environment keeps an emergency local account for exactly this.

1.3 RBAC: grant rights to groups, not people

THEORY

The whole point of central identity is that access follows *group membership*, not individual edits on each box. Put `jdoe` in `linux-admins` once, and every joined host can grant that group elevated rights. Remove the user from the group and access disappears everywhere instantly – this is how real least-privilege scales.

```
# on each host: let the directory group use sudo (drop-in sudoers)
echo '%linux-admins@lab.local ALL=(ALL) ALL' | sudo tee /etc/sudoers.d/domain-admins
# FreeIPA can centralise this instead, so you never touch the host:
ipa sudorule-add admins_all --hostcat=all --cmdcat=all
ipa sudorule-add-user admins_all --groups=linux-admins
```

VERIFY IT WORKS

Add a second domain user who is *not* in `linux-admins`; confirm they can log in but `sudo -l` shows no privileges, while `jdoe` has full sudo. Membership, not per-host edits, decided access.

1.4 Red Team – why the directory is the crown jewel

RED TEAM – ATTACK

From Kali, enumerate the directory the way an attacker who landed one foothold would. Authenticate, then read what the directory will tell any valid user:

```
# enumerate users, groups, and service accounts over LDAP
ldapsearch -x -H ldap://10.10.10.5 -b "dc=lab,dc=local" "(objectClass=user)"
# request service tickets to crack offline (Kerberoasting) -- detection practice
GetUserSPNs.py lab.local/jdoe -dc-ip 10.10.10.5 -request
# map attack paths visually
bloodhound-python -d lab.local -u jdoe -p '<pass>' -c All -ns 10.10.10.5
```

THEORY

What you just learned to watch for. *Kerberoasting* requests tickets for service accounts and cracks them offline; *AS-REP roasting* targets accounts without pre-authentication; *BloodHound* maps the shortest path from any user to domain admin. The defence is detection (these generate distinctive directory events you will alert on in Module 8), strong service-account passwords, and least privilege so one foothold cannot reach the top. This is the heart of enterprise security work, and it only exists once you have a directory.

Tie-in: now that you have a directory, your proxy should authenticate users against it rather than allowing anyone on the subnet. You wire that up in Module 4.

MODULE 2

Networking Foundations (Practical)

You cannot secure what you do not understand. This module covers every core concept your firewall, proxy, and IDS quietly depend on. Each topic gets a short *why*, then a command you run *now* on your `lan-client` to see it for real.

2.1 The OSI model, the practical version

Forget memorising seven layers for an exam. Use the model as a troubleshooting map: when something breaks, you walk *up* the layers until you find where it fails.

L	Layer	You see it as	When it breaks
1	Physical	cable, wifi, link light	no link / no carrier
2	Data Link	MAC address, switch, VLAN	arp fails, wrong VLAN
3	Network	IP address, routing, NAT	ping by IP fails
4	Transport	TCP/UDP ports	port closed / filtered
5-6	Session/Presn.	TLS, encoding	cert errors
7	Application	HTTP, DNS, SSH	app returns an error

VERIFY IT WORKS

Run a layered diagnosis on your client. Each command tests one layer up:

```
ip link show          # L1/L2: is the interface up, do I have a MAC?
ip addr show          # L3: do I have an IP address?
ip route show         # L3: do I have a default gateway?
ping -c2 10.10.10.1   # L3: can I reach the firewall?
ping -c2 1.1.1.1      # L3: can I reach the internet by IP?
nslookup example.com  # L7/DNS: does name resolution work?
curl -I http://example.com # L7: does the app layer respond?
```

The *first* command that fails tells you which layer to fix.

2.2 IP addressing & subnetting

THEORY

An IPv4 address is 32 bits split into a *network* part and a *host* part. The mask (the /24) says how many leading bits are the network. Devices on the same network talk directly; to reach another network they must go through a router (the gateway). That single fact is why the firewall sits between every segment.

Key numbers to internalise (these are the only ones you need 95% of the time):

CIDR	Mask	Hosts	Typical use
/30	255.255.255.252	2	point-to-point link
/29	255.255.255.248	6	tiny segment
/28	255.255.255.240	14	small DMZ
/24	255.255.255.0	254	standard LAN (this lab)
/16	255.255.0.0	65534	large campus

VERIFY IT WORKS

Install `ipcalc` and let the math be done for you, then check by hand:

```
sudo apt install ipcalc -y
ipcalc 10.10.10.0/24      # network, broadcast, host range, mask
ipcalc 10.10.30.0/28      # see how /28 carves a smaller block
```

Confirm: network `.0`, gateway `.1`, broadcast `.255`, usable hosts `.1-.254`.

2.3 TCP vs UDP and ports

THEORY

TCP is a phone call: a handshake (SYN, SYN-ACK, ACK) sets up a connection, data is acknowledged, lost packets are resent. UDP is a postcard: fire and forget, no handshake, no guarantee – used by DNS, DHCP, VoIP, and games where speed beats reliability. A *port* is just a number identifying which service a packet is for: 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 3389 RDP. Firewalls filter largely on the protocol + port pair.

VERIFY IT WORKS

Watch a TCP handshake happen in real time, then see listening ports:

```
# capture the 3-way handshake to a web server
sudo tcpdump -ni any 'tcp port 80 and host example.com' &
curl -I http://example.com
# SYN -> SYN-ACK -> ACK appears in the capture; then kill it:
sudo pkill tcpdump

# what is listening on this machine, TCP and UDP?
ss -tulnp
```

2.4 Routing

THEORY

A routing table answers one question for every packet: "given this destination IP, which interface and next hop do I send it to?" The *default route* (`0.0.0.0/0`) is the catch-all: anything not on a directly connected network goes to the gateway. In your lab the firewall holds the routes that join LAN, DMZ, and WAN.

```
ip route show
# default via 10.10.10.1 dev eth0    <- everything unknown goes to the firewall
# 10.10.10.0/24 dev eth0             <- this network is directly connected
ip route get 1.1.1.1                # ask the kernel which route a packet takes
traceroute 1.1.1.1                  # see each router hop toward the internet
```

2.5 NAT and PAT

THEORY

NAT (Network Address Translation) rewrites IP addresses as packets cross the firewall. **PAT** (Port Address Translation, a.k.a. NAT overload or "masquerade") is the everyday kind: many private hosts (10.10.10.x) share one public WAN address, distinguished by source port. It is why your whole LAN reaches the internet through a single public IP. *Source NAT* hides outbound clients; *Destination NAT* (port forwarding) exposes an inbound service. You configure both on the firewall in Module 3.

Quick mental model: **SNAT** = "who do I look like when I leave?" **DNAT** = "who actually receives traffic aimed at my public IP?"

2.6 DNS

THEORY

DNS turns names into addresses. A client asks a resolver for **example.com**; the resolver walks the hierarchy (root → .com → example.com's nameserver) and returns an A record (IPv4) or AAAA (IPv6). It mostly rides UDP/53, falling back to TCP for large answers. Controlling DNS is a powerful security lever – Module 4 uses it to block sites.

```
dig example.com A +short           # the IP behind a name
dig example.com +trace             # watch the full resolution from the root
dig @10.10.10.1 example.com       # ask YOUR firewall's resolver specifically
```

2.7 DHCP

THEORY

DHCP hands out IP addresses automatically through a four-step exchange you should know by name: **DORA** – Discover, Offer, Request, Acknowledge. The firewall will be your DHCP server for the LAN, leasing addresses from a pool while reserving .1-.9 for static infrastructure.

```
# on the client, watch a lease being obtained
sudo dhclient -v eth0
# inspect the current lease (address, gateway, DNS, lease time)
cat /var/lib/dhcp/dhclient.leases 2>/dev/null || nmcli device show eth0
```

2.8 VLANs

THEORY

A VLAN tags Ethernet frames (802.1Q) so one physical switch behaves like several isolated switches. Devices in VLAN 10 cannot talk to VLAN 30 without passing through a router/-firewall – exactly the segmentation our topology needs. In a virtual lab, each "internal network" is effectively a VLAN.

```
# create a tagged VLAN sub-interface (example: VLAN 30 on eth0)
sudo ip link add link eth0 name eth0.30 type vlan id 30
sudo ip addr add 10.10.30.5/24 dev eth0.30
sudo ip link set eth0.30 up
ip -d link show eth0.30          # the "vlan id 30" line confirms the tag
```

2.9 HTTP and HTTPS

THEORY

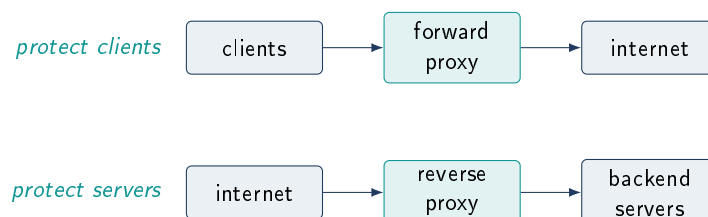
HTTP is plain-text request/response on port 80 – anyone on the path can read it. HTTPS wraps the same HTTP inside TLS on port 443: the server proves identity with a certificate, a key is negotiated, and traffic is encrypted. For security work the implication is huge: a proxy can read and filter HTTP freely, but HTTPS only shows the destination hostname (via SNI) unless you do TLS interception.

```
curl -v http://example.com 2>&1 | head -n 20 # see the plain request/headers
curl -vI https://example.com 2>&1 | grep -i 'subject\|issuer\|TLS' # the cert + TLS
```

2.10 Forward proxy vs reverse proxy

THEORY

A **forward proxy** sits in front of *clients*: every workstation sends its outbound web traffic through it, so you can filter, log, and cache what users reach (this is your Squid box). A **reverse proxy** sits in front of *servers*: the internet talks to it, and it forwards requests to backend servers, adding TLS, load balancing, and a shield (this is Nginx in front of **web-dmz**). Same word, opposite direction – remember which side it protects.



2.11 Firewall logic: stateful vs stateless

THEORY

A **stateless** firewall judges each packet alone against rules – fast but dumb; you must write rules for both directions. A **stateful** firewall remembers connections: if you allow an outbound request, the reply is automatically permitted because it belongs to an *established* connection. Every modern firewall (pfSense, iptables/nftables with conntrack) is stateful, which is why your rules mostly describe *who may start* a conversation.

This single idea simplifies all of Module 3: you allow the *initiating* direction and let the state table handle the return traffic. You will see the literal `ESTABLISHED,RELATED` match in the iptables rules later.

MODULE 3

The Firewall: Routing, NAT & Rules

This is the heart of the lab. You will assign the firewall's interfaces, give the LAN internet access through NAT, then write the full rule set: blocking ports, allowing only specific IPs, forwarding ports, isolating the DMZ, and the per-host allow/deny that proves you understand stateful logic. Concepts are shown both as **pfSense** actions and as the equivalent **iptables/nftables** commands so you understand what the GUI does underneath.

3.1 Bring up WAN and LAN

◎ OBJECTIVE

The firewall has a WAN leg with internet access and a LAN leg at 10.10.10.1/24 serving DHCP, and a LAN client can reach the internet through it.

3.1.1 Assign interfaces (pfSense)

After installing pfSense, the console asks you to assign interfaces. Map them to your virtual NICs:

```
WAN -> em0 (NIC bridged/NAT to host -> gets internet)
LAN -> em1 (NIC on "internal-LAN" network -> 10.10.10.1/24)
OPT1 -> em2 (NIC on "internal-DMZ" network -> 10.10.30.1/24) # add later
```

Then in the web GUI (<https://10.10.10.1> from the LAN client, default admin/pfsense):
Interfaces → **LAN** set static IPv4 10.10.10.1/24; **Services** → **DHCP Server** → **LAN** enable a pool 10.10.10.100-10.10.10.200 (leaving .1-.99 for static infra).

3.1.2 The same idea on a plain Linux router (iptables)

If you ever build the gateway on Ubuntu instead of pfSense, this is the minimum to route + NAT a LAN to the internet:

```
# enable forwarding between interfaces
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-router.conf
sudo sysctl --system

# WAN = eth0 (internet), LAN = eth1 (10.10.10.0/24)
# SOURCE NAT / PAT: the whole LAN shares the WAN IP
sudo iptables -t nat -A POSTROUTING -o eth0 -s 10.10.10.0/24 -j MASQUERADE
```

VERIFY IT WORKS

From lan-client: `ip route` shows default via 10.10.10.1, and `ping -c2 1.1.1.1` succeeds. You now have routing + NAT working.

3.2 Stateful baseline: the default-deny posture

THEORY

A good firewall *denies by default* and explicitly allows what is needed. Because the firewall is stateful, you only permit the side that *starts* a connection; replies are matched as ESTABLISHED,RELATED and let through automatically.

The classic iptables baseline (read it top to bottom – order matters):

```
# flush, then set default-deny on the FORWARD chain
sudo iptables -F FORWARD
sudo iptables -P FORWARD DROP

# 1) always allow replies to connections we already permitted
sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# 2) let the LAN start connections out to the internet
sudo iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
# everything else hits the default DROP
```

3.3 Block specific ports

◎ OBJECTIVE

Stop the LAN from using Telnet (23) and SMB (445) outbound, while leaving web browsing intact.

```
# insert blocks ABOVE the broad "LAN out" allow rule
sudo iptables -I FORWARD -i eth1 -p tcp --dport 23 -j REJECT
sudo iptables -I FORWARD -i eth1 -p tcp --dport 445 -j REJECT
```

In pfSense the same is **Firewall** → **Rules** → **LAN**: a Block rule, protocol TCP, destination port 23, placed above the allow rule (pfSense evaluates top-down, first match wins).

VERIFY IT WORKS

From the client: `nc -vz <internet-host> 23` times out / is refused, but `curl -I http://example.com` still works.

3.4 Allow only certain source IPs

◎ OBJECTIVE

Only the admin workstation 10.10.10.10 may reach the firewall's management GUI; every other host is denied.

```
# allow admin host to the firewall's web GUI (port 443 on the firewall itself)
sudo iptables -A INPUT -p tcp -s 10.10.10.10 --dport 443 -j ACCEPT
# deny all other hosts to the GUI
sudo iptables -A INPUT -p tcp --dport 443 -j DROP
```

THEORY

This is an **allow-list** (default deny, name the exceptions). Allow-lists are strictly safer than deny-lists because anything you forgot is blocked, not exposed. Prefer them for management interfaces.

3.5 Permit one machine, deny another

◎ OBJECTIVE

On the LAN, 10.10.10.10 may reach the DMZ web server, but 10.10.10.11 may not – proving you can do per-host policy.

```
sudo iptables -A FORWARD -s 10.10.10.10 -d 10.10.30.10 -p tcp --dport 80 -j ACCEPT
sudo iptables -A FORWARD -s 10.10.10.11 -d 10.10.30.10 -j REJECT
```

VERIFY IT WORKS

From .10: `curl http://10.10.30.10` returns the page. From .11: the same command is refused. Per-host policy confirmed.

3.6 Allow only HTTP (a tightly scoped policy)

◎ OBJECTIVE

A locked-down segment where hosts may only make outbound HTTP/HTTPS – nothing else leaves.

```
sudo iptables -F FORWARD
sudo iptables -P FORWARD DROP
sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# DNS is required for web to work at all
sudo iptables -A FORWARD -i eth1 -p udp --dport 53 -j ACCEPT
# only HTTP + HTTPS may initiate outbound
sudo iptables -A FORWARD -i eth1 -p tcp --dport 80 -j ACCEPT
sudo iptables -A FORWARD -i eth1 -p tcp --dport 443 -j ACCEPT
# (everything else falls through to DROP)
```

3.7 Configure NAT and port forwarding

◎ OBJECTIVE

Expose the DMZ web server to the "internet" so that hitting the firewall's WAN IP on port 80 reaches 10.10.30.10:80 – this is destination NAT (DNAT).

```
# DNAT: WAN:80 -> web-dmz:80
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 \
    -j DNAT --to-destination 10.10.30.10:80
# allow the forwarded traffic through the filter
```

```
sudo iptables -A FORWARD -p tcp -d 10.10.30.10 --dport 80 -j ACCEPT
```

In pfSense: **Firewall** → **NAT** → **Port Forward**, WAN, TCP, dest port 80, redirect target 10.10.30.10:80; pfSense offers to auto-create the matching firewall rule – accept it.

VERIFY IT WORKS

From attacker-kali on the WAN side: `curl http://<firewall-WAN-IP>` returns the DMZ server's page. Inbound port forwarding works.

3.8 Block SSH from the internet

◎ OBJECTIVE

SSH (22) must *never* be reachable from the WAN, even though it is fine inside the LAN.

```
# explicitly drop SSH arriving on the WAN interface, to the firewall or forwarded
sudo iptables -A INPUT -i eth0 -p tcp --dport 22 -j DROP
sudo iptables -A FORWARD -i eth0 -p tcp --dport 22 -j DROP
```

THEORY

Exposing SSH to the internet is one of the most common ways small networks get owned (constant automated brute-force). The right pattern is: SSH only from inside, or only via a **bastion host** / VPN. You build both later (bastion in Module 9, and Fail2Ban in Module 7 for any SSH you do expose internally).

VERIFY IT WORKS

From Kali (WAN): `ssh user@<firewall-WAN-IP>` hangs/refused. From lan-client: `ssh user@10.10.10.20` still works.

3.9 DMZ-style segmentation

◎ OBJECTIVE

The DMZ web server can be reached from the WAN (via the port-forward above) and can answer DNS, but it **cannot initiate** connections into the trusted LAN – so if it is compromised, the attacker is contained.

```
# DMZ may go out to the internet for updates...
sudo iptables -A FORWARD -i eth2 -o eth0 -j ACCEPT
# ...but must NOT reach the trusted LAN
sudo iptables -A FORWARD -i eth2 -o eth1 -j DROP
# LAN may reach DMZ services (managed above per-host)
```



WATCH OUT

The single most important DMZ rule is the *DMZ→LAN deny*. If you forget it, a hacked web server becomes a pivot straight into your trusted machines. Test it explicitly: from **web-dmz** try `ping 10.10.10.10` – it must fail.

3.10 Make iptables rules survive a reboot

```
sudo apt install iptables-persistent -y
sudo netfilter-persistent save      # writes /etc/iptables/rules.v4
```

MODULE 4

Proxies: Filtering, Logging & Control

Now you control *what* the network may reach, not just whether it has connectivity. The forward proxy (Squid) governs outbound web access for clients; the reverse proxy (Nginx) shields the DMZ web server. Both give you the logs that feed the SIEM later.

4.1 Forward proxy with Squid

◎ OBJECTIVE

proxy-squid (10.10.10.20) proxies all LAN web traffic, with access rules, URL/domain filtering, and full request logging.

```
sudo apt update && sudo apt install squid -y
sudo cp /etc/squid/squid.conf /etc/squid/squid.conf.bak # always keep a backup
```

A minimal but real `/etc/squid/squid.conf`:

```
# --- who is allowed to use this proxy ---
acl lan_net src 10.10.10.0/24
http_access allow lan_net
http_access deny all

# listen on the LAN interface
http_port 10.10.10.20:3128

# --- access logging (this is your audit trail) ---
access_log /var/log/squid/access.log squid
```

```
sudo systemctl restart squid && sudo systemctl enable squid
```

4.1.1 Point a client at the proxy

```
# test it explicitly from lan-client
curl -x http://10.10.10.20:3128 -I http://example.com
# system-wide (bash session):
export http_proxy=http://10.10.10.20:3128
export https_proxy=http://10.10.10.20:3128
```

4.2 URL / domain filtering: restrict websites

◎ OBJECTIVE

Block YouTube and a list of other domains for everyone using the proxy.

Create a blocklist file and reference it from the config:

```
sudo tee /etc/squid/blocked_sites.txt >/dev/null <<'EOF'
.youtube.com
.googlevideo.com
.facebook.com
.tiktok.com
EOF
```

Add to `squid.conf` *above* the allow rule:

```
acl blocked_domains dstdomain "/etc/squid/blocked_sites.txt"
http_access deny blocked_domains
http_access allow lan_net
http_access deny all
```

```
sudo systemctl reload squid
```

THEORY

The leading dot (`.youtube.com`) matches the domain *and all subdomains*. Blocking YouTube genuinely needs the video CDN (`googlevideo.com`) too – a great lesson in how one "site" is really many domains. For HTTPS, Squid sees the hostname via SNI/CONNECT, which is enough to block by domain without decrypting.

VERIFY IT WORKS

From the client through the proxy, the YouTube request is refused with **403 Forbidden** while `example.com` still loads:

```
curl -x http://10.10.10.20:3128 -I https://www.youtube.com # -> 403 Forbidden
curl -x http://10.10.10.20:3128 -I http://example.com      # -> still works
```

Then check the log shows the denial:

```
sudo tail -f /var/log/squid/access.log # look for TCP_DENIED / 403 entries
```

4.3 Logging requests (your audit trail)

Every line in `access.log` records timestamp, client IP, result code, bytes, method, and URL. This is gold for investigations and feeds the dashboard later.

```
# top 10 destinations requested by the LAN today
sudo awk '{print $7}' /var/log/squid/access.log | sort | uniq -c | sort -rn | head
# everything one host requested
sudo grep '10.10.10.11' /var/log/squid/access.log
```


4.4 Force traffic through the proxy (transparent proxy)

THEORY

An *explicit* proxy can be bypassed by a user who simply unsets the proxy setting. A **transparent** (intercepting) proxy can't: the firewall redirects all outbound port-80 traffic to Squid whether the client likes it or not. This is how you actually *enforce* the policy.

OBJECTIVE

The firewall transparently redirects LAN web traffic to Squid, so filtering applies even with no proxy configured on the client.

On the firewall (iptables), redirect LAN HTTP to the proxy box; configure Squid with an intercept port:

```
# on the firewall: send LAN port-80 traffic to Squid's intercept port
sudo iptables -t nat -A PREROUTING -i eth1 -p tcp --dport 80 \
-j DNAT --to-destination 10.10.10.20:3129
```

```
# add to squid.conf: an interception port (no client config needed)
http_port 10.10.10.20:3129 intercept
```

VERIFY IT WORKS

On the client, *remove* any proxy setting (`unset http_proxy`) then browse a blocked site – it is *still* blocked. Enforcement, not just suggestion.

WATCH OUT

Transparent interception of HTTPS requires installing a trusted CA on every client (TLS interception) and has real privacy implications. In a learning lab it is fine and instructive; in the real world it is a policy/legal decision. Stick to HTTP interception + SNI-based HTTPS domain blocking unless you specifically want to practise TLS interception.

4.5 Reverse proxy with Nginx (protect the DMZ server)

OBJECTIVE

Nginx on `web-dmz` sits in front of the application, terminates TLS, hides the backend, and logs every request – the public face of your DMZ.

```
sudo apt install nginx -y
```

```
# /etc/nginx/sites-available/app (reverse proxy to a backend on :8080)
server {
    listen 80;
    server_name web.lab.local;

    access_log /var/log/nginx/app_access.log;

    location / {
```

```

    proxy_pass          http://127.0.0.1:8080;
    proxy_set_header    Host $host;
    proxy_set_header    X-Real-IP $remote_addr;
    proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
}
}

```

```

sudo ln -s /etc/nginx/sites-available/app /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx      # -t validates before reload

```

THEORY

Why a reverse proxy? It gives you one controlled entry point: TLS termination, rate limiting, hiding backend versions, load balancing across multiple app servers, and a natural place to bolt on a Web Application Firewall (you do exactly that in the cloud module). The backend never talks to the internet directly.

VERIFY IT WORKS

From Kali via the firewall's port-forward: `curl -I http://<firewall-WAN-IP>` shows **Server: nginx** and the request appears in `/var/log/nginx/app_access.log` – but the backend on `:8080` is never directly reachable from outside.

4.6 TLS interception (SSL Bump)

THEORY

Until now Squid handles HTTPS with a blind **CONNECT** tunnel: it sees only the destination hostname (via SNI) and passes the encrypted bytes through untouched. That is enough to *block* a domain but not to *inspect* it – and modern malware, data exfiltration, and command-and-control all hide inside that encryption. **SSL Bump** makes Squid terminate the client's TLS, decrypt, inspect, then re-encrypt to the real server using its own Certificate Authority. The client must trust that CA, which is why you install it on managed machines.

◎ OBJECTIVE

Squid decrypts outbound HTTPS from `lan-client` using a proxy-generated CA, while *splicing* (passing through untouched) a list of sensitive or pinned domains.

4.6.1 Generate the proxy CA and configure bumping

```

# create the proxy's own Certificate Authority
sudo openssl req -new -newkey rsa:2048 -days 3650 -nodes -x509 \
  -keyout /etc/squid/bump.key -out /etc/squid/bump.crt \
  -subj "/CN=Lab Squid Proxy CA"
# initialise Squid's certificate cache (generates per-site certs on the fly)
sudo /usr/lib/squid/security_file_certgen -c -s /var/lib/squid/ssl_db -M 4MB
sudo chown -R proxy:proxy /var/lib/squid/ssl_db

```

```

# add to squid.conf
http_port 10.10.10.20:3130 ssl-bump \
  cert=/etc/squid/bump.crt key=/etc/squid/bump.key \

```

```
generate-host-certificates=on
sslcrttd_program /usr/lib/squid/security_file_certgen -s /var/lib/squid/ssl_db -M 4MB

# splice (do NOT decrypt) banking and other pinned/sensitive domains
acl no_bump ssl::server_name_regex "/etc/squid/no_bump.txt"
ssl_bump splice no_bump
# everything else: peek at the handshake, then bump (decrypt)
ssl_bump peek all
ssl_bump bump all
```

4.6.2 Trust the CA on the client

```
# copy bump.crt to the client, then trust it system-wide
sudo cp bump.crt /usr/local/share/ca-certificates/lab-proxy.crt
sudo update-ca-certificates
# browsers with their own trust store (Firefox) need it imported separately
```

VERIFY IT WORKS

Through the bumping port, the certificate the client sees is now *issued by your proxy CA*, proving interception, while a spliced banking domain still presents its real certificate:

```
curl -x http://10.10.10.20:3130 -v https://example.com 2>&1 | grep -i issuer
# issuer: CN=Lab Squid Proxy CA          <- decrypted/inspected
curl -x http://10.10.10.20:3130 -v https://yourbank.example 2>&1 | grep -i
issuer
# issuer: <real bank CA>                  <- spliced, untouched
```

WATCH OUT

You cannot bump everything, and that is the real lesson. Certificate pinning (banking apps, Chrome's pinned Google certificates, anything using HSTS preload) will simply break when intercepted – hence the splice list. The professional pattern is *bump-and-splice*: decrypt what you can, bypass what pins. Also note the privacy and legal weight of decrypting users' traffic; in the real world this is a documented policy decision, not a technical default. And the next frontier, Encrypted Client Hello, hides even the SNI you rely on for splicing.

4.7 Authenticate the proxy against the directory

THEORY

Right now anyone on 10.10.10.0/24 may use the proxy. With the directory from Module 1, you can require users to *authenticate*, so the access log records a username (not just an IP) and policy can follow group membership.

```
# squid.conf: Kerberos (single sign-on) against the directory
auth_param negotiate program /usr/lib/squid/negotiate_kerberos_auth -s GSS_C_NO_NAME
auth_param negotiate children 10
acl authenticated proxy_auth REQUIRED
http_access allow authenticated
http_access deny all
```

VERIFY IT WORKS

A request now requires a valid domain ticket, and `access.log` shows the *username* responsible for each request – accountability, not just an IP address. Try without a ticket: the proxy returns `407 Proxy Authentication Required`.

MODULE 5

VPN: Secure Remote Access

You blocked SSH from the internet in Module 3. So how do you get in remotely? Through an encrypted VPN tunnel. This module builds a VPN gateway on the firewall, the modern way (WireGuard) and the classic way (OpenVPN), so you understand both.

THEORY

A VPN creates an encrypted tunnel between a remote device and your network. Instead of exposing each service (SSH, the GUI, internal apps) to the internet, you expose *one* hardened VPN port; once connected, the remote device behaves as if it were on the LAN. This collapses your internet-facing attack surface to a single, strongly authenticated entry point. There are two families: **IPsec** (standard, heavier), and **SSL/TLS-based** (OpenVPN) and the newer **WireGuard** (tiny, fast, modern crypto).

5.1 WireGuard (modern, recommended)

🎯 OBJECTIVE

A WireGuard tunnel from **attacker-kali** (acting as your "remote laptop") into the LAN, so you can reach 10.10.10.x services without exposing them publicly.

5.1.1 Server side (the firewall / a Linux gateway)

```
sudo apt install wireguard -y
# generate the server keypair
wg genkey | sudo tee /etc/wireguard/server_private.key | wg pubkey | sudo tee /etc/
wireguard/server_public.key
```

```
# /etc/wireguard/wg0.conf (server)
[Interface]
Address      = 10.200.0.1/24
ListenPort   = 51820
PrivateKey   = <SERVER_PRIVATE_KEY>
# route VPN clients into the LAN and NAT them out
PostUp       = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o
               eth1 -j MASQUERADE
PostDown     = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o
               eth1 -j MASQUERADE

[Peer]
PublicKey    = <CLIENT_PUBLIC_KEY>
AllowedIPs   = 10.200.0.2/32
```

```
sudo wg-quick up wg0
sudo systemctl enable wg-quick@wg0
```

5.1.2 Client side

```
wg genkey | tee client_private.key | wg pubkey | tee client_public.key
```

```
# client wg0.conf
[Interface]
Address      = 10.200.0.2/24
PrivateKey   = <CLIENT_PRIVATE_KEY>

[Peer]
PublicKey    = <SERVER_PUBLIC_KEY>
Endpoint     = <FIREWALL_WAN_IP>:51820
AllowedIPs   = 10.10.10.0/24, 10.200.0.0/24
PersistentKeepalive = 25
```

VERIFY IT WORKS

Bring the tunnel up with `sudo wg-quick up wg0`, then run `sudo wg` to see a handshake and transfer counters. Now `ssh user@10.10.10.20` works *through the tunnel*, even though SSH is firewalled off from the raw internet. That is the whole point.

5.2 OpenVPN (the classic)

For breadth, stand up OpenVPN too – many enterprises still run it, and pfSense ships an OpenVPN wizard.

```
# fastest path on a Linux gateway: the well-known installer script approach
sudo apt install openvpn easy-rsa -y
# build a CA + server cert with easy-rsa, then issue client certs:
make-cadir ~/openvpn-ca && cd ~/openvpn-ca
./easyrsa init-pki
./easyrsa build-ca nopass
./easyrsa gen-req server nopass && ./easyrsa sign-req server server
./easyrsa gen-req client1 nopass && ./easyrsa sign-req client client1
```

In pfSense this is all GUI: **VPN → OpenVPN → Wizard** creates the CA, server, and exportable client configs for you. Use the GUI to *understand the moving parts* (CA, server cert, client cert, TLS auth key), then appreciate how much WireGuard simplifies by comparison.

THEORY

WireGuard vs OpenVPN, the short version: WireGuard is ~4000 lines of code, uses fixed modern crypto, connects almost instantly, and is easy to audit. OpenVPN is far larger, highly configurable, runs over TCP or UDP (useful to evade restrictive networks), and has a longer enterprise track record. Learn WireGuard for new builds; recognise OpenVPN because you *will* meet it in the field.

MODULE 6

IDS/IPS & Attack Simulation

Time to attack your own lab. You will install Suricata to watch traffic, then run the three classic attack categories from Kali – scanning, brute-forcing, and web scanning – and confirm each one generates alerts and (where configured) is blocked. **Only ever run these against your own lab machines.**

6.1 Install Suricata (IDS mode first)

◎ OBJECTIVE

`ids-sensor` (10.10.30.50) inspects DMZ traffic and raises alerts in `eve.json`. In IDS mode it watches; in IPS mode it can drop.

```
sudo apt install suricata -y
# point it at your monitored interface and pull the community ruleset
sudo suricata-update
sudo sed -i 's/^\( *\)- interface: .*/\1- interface: eth0/' /etc/suricata/suricata.
    yaml
sudo systemctl restart suricata && sudo systemctl enable suricata
```

Add a simple custom rule so you can see detection working immediately:

```
# /etc/suricata/rules/local.rules
alert icmp any any -> $HOME_NET any (msg:"LAB ICMP ping detected"; sid:1000001; rev
:1;)
alert tcp any any -> $HOME_NET 22 (msg:"LAB SSH connection attempt"; sid:1000002; rev
:1;)
```

```
# ensure local.rules is loaded, then restart
grep -q local.rules /etc/suricata/suricata.yaml || \
    sudo sed -i '/rule-files:/a\ - local.rules' /etc/suricata/suricata.yaml
sudo systemctl restart suricata
```

VERIFY IT WORKS

Ping the sensor from another box, then watch alerts arrive:

```
sudo tail -f /var/log/suricata/eve.json | grep --line-buffered '"event_type":'
    alert"
```

You should see your "LAB ICMP ping detected" alert appear in real time.

6.2 Attack 1 – Port scanning (reconnaissance)

RED TEAM – ATTACK

From Kali, map what the DMZ server exposes. This is what every attacker does first.

```
nmap -sS -sV 10.10.30.10      # SYN scan + service/version detection
nmap -p- 10.10.30.10         # all 65535 ports
nmap -sV --script vuln 10.10.30.10 # basic vuln scripts
```

VERIFY IT WORKS

Defender view: on the firewall the scan shows as a burst of connections to many ports – `sudo tail -f /var/log/syslog | grep DROP` (if you log drops). Suricata's ruleset flags it as a port scan in `eve.json`. You just saw recon from both sides.

6.3 Attack 2 – SSH brute force

RED TEAM – ATTACK

Hammer SSH on an internal target with a wordlist (set up a deliberately weak test account first – never your real one).

```
hydra -l testuser -P /usr/share/wordlists/rockyou.txt \
ssh://10.10.10.20 -t 4
```

VERIFY IT WORKS

Defender view: a flood of failed logins in `/var/log/auth.log` on the target, and your Suricata "SSH connection attempt" alerts firing repeatedly. In Module 7 you add Fail2Ban so this attack gets the source IP *banned* automatically – and in Module 8 you watch the full chain end to end.

6.4 Attack 3 – Web scanning

RED TEAM – ATTACK

Probe the DMZ web server for misconfigurations, hidden paths, and known issues.

```
nikto -h http://10.10.30.10      # known web flaws & headers
gobuster dir -u http://10.10.30.10 \
-w /usr/share/wordlists/dirb/common.txt # hidden directories/files
```

VERIFY IT WORKS

Defender view: Nginx's `access.log` fills with hundreds of rapid 404s from one IP – the signature of directory brute-forcing. Suricata flags the scanner's user-agent. This is exactly why a WAF (cloud module) exists.

6.5 Turn Suricata into an IPS (block, not just alert)

THEORY

IDS detects and alerts. **IPS** sits *inline* and can drop malicious packets. The difference is one of placement: an IDS taps a copy of traffic; an IPS is in the path so it can act. Promote rules from **alert** to **drop** and run Suricata inline (via NFQUEUE) to make it enforce.

```
# send forwarded traffic to Suricata's queue, then run it inline
sudo iptables -I FORWARD -j NFQUEUE --queue-num 0
sudo suricata -q 0 -c /etc/suricata/suricata.yaml &
```

Change a rule action from **alert** to **drop** to actively block:

```
drop tcp any any -> $HOME_NET 22 (msg:"LAB block SSH brute force"; \
    flow:to_server; threshold:type both, track by_src, count 5, seconds 30; \
    sid:1000010; rev:1;)
```

VERIFY IT WORKS

Re-run the Hydra brute force from Kali: after a few attempts the connections are *dropped* by the IPS, not merely logged. You converted detection into prevention.

WATCH OUT

Run IPS mode in the lab only. An overly aggressive inline rule can block legitimate traffic (a "false positive" outage). The professional workflow is: run in IDS mode first, tune rules until alerts are clean, *then* promote to drop.

6.6 On-premises WAF (ModSecurity + OWASP CRS)

THEORY

You gave the cloud DMZ a WAF in Module 9, but the on-prem web server is still naked behind a plain reverse proxy: the **nikto** and **gobuster** scans slid straight through to the backend. A **Web Application Firewall** inspects HTTP *content* – request bodies, parameters, headers – and blocks attack patterns (SQL injection, cross-site scripting, path traversal) before they reach the app. **ModSecurity** is the open-source engine; the **OWASP Core Rule Set (CRS)** is the community signature pack it runs.

🎯 OBJECTIVE

The Nginx reverse proxy on **web-dmz** runs ModSecurity with the OWASP CRS, and your Module 6 web attacks are now dropped at the proxy instead of reaching the backend.

```
# build the ModSecurity v3 engine + the Nginx connector, then fetch the CRS
sudo apt install libmodsecurity3 nginx-module-modsecurity -y # distro-dependent
git clone https://github.com/coreruleset/coreruleset /etc/nginx/modsec/crs
sudo cp /etc/nginx/modsec/crs/crs-setup.conf.example \
    /etc/nginx/modsec/crs/crs-setup.conf
```

```
# /etc/nginx/modsec/main.conf
```

```

Include /etc/modsecurity/modsecurity.conf
Include /etc/nginx/modsec/crs/crs-setup.conf
Include /etc/nginx/modsec/crs/rules/*.conf

```

Enable it inside the reverse-proxy server block from Module 4:

```

server {
    listen 80;
    server_name web.lab.local;
    modsecurity on;
    modsecurity_rules_file /etc/nginx/modsec/main.conf;
    location / { proxy_pass http://127.0.0.1:8080; }
}

```

```
sudo nginx -t && sudo systemctl reload nginx
```

6.6.1 Tune before you block (the real WAF lesson)

WATCH OUT

A WAF in blocking mode *will* reject legitimate traffic until tuned – the same false-positive problem you met promoting Suricata from IDS to IPS. Run ModSecurity in **DetectionOnly** first, watch what it *would* have blocked, raise/lower the CRS *paranoia level* and add exceptions, *then* switch to blocking. Set `SecRuleEngine DetectionOnly` in `modsecurity.conf` to start.

RED TEAM – ATTACK

Re-run the exact web attacks from earlier in this module and watch the difference:

```

nikto -h http://10.10.30.10
# classic injection probes the CRS should catch:
curl "http://10.10.30.10/?id=1' OR '1'='1" # SQL injection
curl "http://10.10.30.10/?q=<script>alert(1)</script>" # XSS

```

VERIFY IT WORKS

With CRS in blocking mode the injection requests return 403 **Forbidden** and the audit log records the matched rule:

```
sudo tail -f /var/log/modsec_audit.log # shows rule ID, matched data, score
```

The same attacks that previously reached the backend now die at the proxy. Defence in depth in action.

Two honest caveats. The Nginx connector for ModSecurity has historically been less actively maintained than the Apache one; **OWASP Coraza** is a modern Go reimplementation that runs the same CRS if you hit friction. And a WAF buys time – it is not a substitute for fixing the vulnerable application underneath.

MODULE 7

Host Hardening

A network is only as strong as the machines on it. This module hardens a Linux host the way you would a real server: lock down SSH, run a host firewall, auto-ban attackers, keep it patched, and enforce least privilege and sane file permissions.

7.1 SSH hardening

🎯 OBJECTIVE

SSH on `proxy-squid` accepts only key-based logins, refuses root, and exposes the smallest possible surface.

Edit `/etc/ssh/sshd_config`:

```
PermitRootLogin no           # never log in directly as root
PasswordAuthentication no    # keys only -- defeats brute force entirely
PubkeyAuthentication yes
AllowUsers admin             # only this account may SSH in
MaxAuthTries 3
LoginGraceTime 20
X11Forwarding no
Protocol 2
```

```
# create a key on your client FIRST, then copy it, THEN disable passwords
ssh-keygen -t ed25519 -C "lab-admin"
ssh-copy-id admin@10.10.10.20
sudo sshd -t && sudo systemctl restart ssh # -t validates config before restart
```

⚠️ WATCH OUT

Always copy your key and confirm key login works in a *second* terminal *before* you set `PasswordAuthentication no` and restart. Otherwise you can lock yourself out. This mistake is a rite of passage – avoid it.

✅ VERIFY IT WORKS

`ssh root@10.10.10.20` is refused; `ssh admin@10.10.10.20` works via key with no password prompt. Surface minimised.

7.2 Host firewall with ufw (and the iptables underneath)

📖 THEORY

`ufw` ("uncomplicated firewall") is a friendly front-end that *writes iptables rules* for you. Use `ufw` for quick host policy; reach for raw `iptables/nftables` when you need the fine control you practised in Module 3.

```
sudo ufw default deny incoming      # default-deny inbound
sudo ufw default allow outgoing
sudo ufw allow from 10.10.10.0/24 to any port 22 proto tcp  # SSH from LAN only
sudo ufw allow 3128/tcp             # the Squid proxy port
sudo ufw enable
sudo ufw status verbose             # review the resulting policy
```

7.3 Fail2Ban: auto-ban attackers

◎ OBJECTIVE

Repeated failed SSH logins get the source IP banned automatically – the defence that answers Module 6's brute force.

```
sudo apt install fail2ban -y
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

```
# /etc/fail2ban/jail.local
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 5

[sshd]
enabled = true
port    = 22
```

```
sudo systemctl restart fail2ban
sudo fail2ban-client status sshd    # see currently banned IPs and totals
```

VERIFY IT WORKS

Re-run the Hydra SSH brute force from Kali. After 5 failures within the window, `sudo fail2ban-client status sshd` lists Kali's IP under "Banned IP list", and further attempts time out. Attack defeated automatically.

7.4 Patch management

THEORY

Most real breaches exploit *known* vulnerabilities that a patch already fixed. Keeping systems updated is unglamorous and the highest-value control you own.

```
sudo apt update && sudo apt upgrade -y      # manual baseline
# automate security updates so you never forget:
sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure -plow unattended-upgrades
# audit what is currently outstanding
apt list --upgradable 2>/dev/null
```

7.5 Least privilege & secure permissions

THEORY

Least privilege: every user/service gets only the access it needs, nothing more. A compromised low-privilege account should not be able to touch the whole system. In practice: no daily use of root, scoped `sudo`, dedicated service accounts.

```
# a service account that cannot log in interactively
sudo useradd -r -s /usr/sbin/nologin svc-app
# grant ONLY the specific command a user needs via sudo (not full root)
echo 'deploy ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx' \
| sudo tee /etc/sudoers.d/deploy
```

Secure file permissions

```
# the permission model: owner / group / others (read=4 write=2 exec=1)
chmod 600 ~/.ssh/id_ed25519 # private key: owner read/write only
chmod 644 /etc/nginx/nginx.conf # config: world-readable, owner-writable
chmod 700 ~/.ssh # dir: owner only
# find world-writable files (a classic misconfiguration to hunt down)
sudo find / -xdev -type f -perm -0002 2>/dev/null
# find SUID binaries (privilege-escalation surface to review)
sudo find / -xdev -perm -4000 -type f 2>/dev/null
```

VERIFY IT WORKS

`ls -l ~/.ssh` shows 600 on the private key (SSH refuses to use a key with loose permissions – a built-in safety check). The world-writable search returns nothing in your config directories.

7.6 Fleet management with Ansible

THEORY

Hardening one host by hand is fine; doing it on every host, consistently, and *proving* it stayed that way is the actual job. **Ansible** is an agentless configuration-management tool: it connects over SSH (the keys you already set up) from a control node on the management network and enforces a described state across the whole fleet. The shift here is from "I did it once" to "it is defined as code and re-applied idempotently" – the move into infrastructure-as-code.

◎ OBJECTIVE

A control node on 10.10.99.0/24 that audits patch levels and pushes the hardened SSH config from this module to the proxy, web server, and client in one command.

```
sudo apt install ansible -y
```

```
# inventory.ini -- the fleet
```

```
[web]
10.10.30.10
[proxy]
10.10.10.20
[clients]
10.10.10.10

[all:vars]
ansible_user=admin
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

A playbook that audits updates and enforces hardened SSH everywhere:

```
# harden.yml
- hosts: all
  become: true
  tasks:
    - name: Report how many security updates are pending
      command: apt list --upgradable
      register: upd
      changed_when: false
    - name: Apply all security updates
      apt: { upgrade: dist, update_cache: yes }
    - name: Enforce hardened sshd settings
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "{{ item.re }}"
        line: "{{ item.line }}"
      loop:
        - { re: '^#?PermitRootLogin', line: 'PermitRootLogin no' }
        - { re: '^#?PasswordAuthentication', line: 'PasswordAuthentication no' }
      notify: restart ssh
  handlers:
    - name: restart ssh
      service: { name: ssh, state: restarted }
```

```
ansible all -i inventory.ini -m ping          # confirm the control node reaches every
host
ansible-playbook -i inventory.ini harden.yml
```

VERIFY IT WORKS

Run the playbook a second time: the SSH tasks report **ok** (not **changed**) because the state already matches – that is *idempotence*, the property that makes config-as-code safe to re-run. Manually loosen **PermitRootLogin** on one host, re-run, and watch Ansible put it back.

7.7 Vulnerability scanning (find what to patch)

THEORY

Patching is only half a loop – you also need to *discover* what is vulnerable. A vulnerability scanner probes hosts and reports known CVEs with severity, so patching becomes prioritised rather than blind. **Find** → **remediate** → **verify** is the real patch-management cycle, and it pairs naturally with the Ansible remediation above.

🕒 OBJECTIVE

An OpenVAS/Greenbone scanner on the management network produces a CVE report for the fleet that drives what your Ansible playbook patches next.

```
# Greenbone Community Edition is the most accessible open-source scanner
sudo apt install openvas -y
sudo gvm-setup          # downloads the vulnerability feeds (takes a while)
sudo gvm-check-setup    # confirm the install is healthy
# then via the web UI: create a target (your subnets) and launch a scan
```

VERIFY IT WORKS

Scan **web-dmz**, find a reported CVE, patch it with the Ansible playbook, then *re-scan* and confirm the finding is gone. You have closed the full loop: discovered, remediated as code, verified.

Scope it like the offensive modules: only scan hosts you own on the lab network. Vulnerability scanners are noisy and intrusive – pointing one at infrastructure you do not control is the same boundary issue as the Module 6 attacks.

7.7.1 Prioritise: you cannot patch everything**THEORY**

A scan returns hundreds of findings; the real skill is deciding *what to fix first*. **CVSS** scores a vulnerability's intrinsic severity 0–10, but severity alone is a trap – a 9.8 nobody can reach matters less than a 7.5 on your internet-facing box. Modern prioritisation adds *exploitability and exposure*: **EPSS** (the probability a CVE will actually be exploited) and CISA's **KEV** catalogue (vulnerabilities being exploited *right now*). The enterprise rule of thumb: anything on KEV, or high-CVSS + high-EPSS *and* internet-reachable, jumps the queue regardless of the raw number.

```
# triage a finding the way a real team does:
# 1) CVSS base score      -> how bad if exploited?      (e.g. nvd.nist.gov)
# 2) EPSS probability     -> how likely to be exploited? (api.first.org/epss)
# 3) on CISA KEV list?    -> being exploited in the wild RIGHT NOW
# 4) is the asset exposed + business-critical?         -> your context
# rank by (exploitability x exposure x impact), not by CVSS alone
```

THEORY

The remediation lifecycle closes around this: *discover* (scan), *prioritise* (above), *remediate* (patch via Ansible, or mitigate with a firewall/WAF rule if no patch exists yet), *verify* (re-scan), and *track* with an SLA – e.g. KEV/critical in 72 hours, high in 30 days. Reporting the trend (open findings by severity over time) is what management actually asks a security engineer for.

VERIFY IT WORKS

Take three findings from your scan and rank them by the four-factor model above rather than CVSS alone – then justify the order in one sentence each. That justification *is* the interview answer to "you have 200 vulns and time for 10, what do you do?"

7.8 Multi-factor authentication on SSH and VPN

THEORY

Keys and Fail2Ban defeat brute force, but a stolen private key still gets an attacker in. **MFA** adds a second factor (a time-based one-time code) so a compromised key alone is not enough – the single highest-leverage control against credential theft, and a near-universal real-world requirement that the base lab omitted.

```
# TOTP for SSH via Google Authenticator PAM module
sudo apt install libpam-google-authenticator -y
google-authenticator          # run as the user; scan the QR with an authenticator app
```

```
# /etc/pam.d/ssh -> add:
auth required pam_google_authenticator.so
# /etc/ssh/ssh_config -> require BOTH key and code:
AuthenticationMethods publickey,keyboard-interactive
KbdInteractiveAuthentication yes
```

VERIFY IT WORKS

Reconnect: after the key is accepted you are prompted for the 6-digit code, and login fails without it. For the VPN, WireGuard's keypair is already a strong factor; for OpenVPN, layer the same TOTP via its PAM plugin. Two factors, not one.

MODULE 8

Integrated Detection: Attack to Dashboard

Individual controls are good; a *pipeline* is what real defenders run. Here you centralise logs into a dashboard and trace two complete attack-to-defence chains, end to end. This module is where the whole lab clicks together.

8.1 Build the dashboard (SIEM)

◎ OBJECTIVE

ids-sensor runs a SIEM that ingests firewall logs, SSH auth logs, Suricata alerts, and proxy logs into one searchable dashboard.

The fastest capable option is **Wazuh** (agent-based, ships with a dashboard); **Security Onion** is the all-in-one alternative if you want a dedicated appliance VM.

```
# Wazuh single-node install (manager + indexer + dashboard)
curl -s0 https://packages.wazuh.com/4.x/wazuh-install.sh
sudo bash wazuh-install.sh -a
# then install the Wazuh agent on each host you want to monitor:
#   proxy-squid, web-dmz, the firewall -> they ship logs to the manager
```

If resources are tight, a lighter pipeline is **rsyslog** forwarding all hosts' logs to the sensor, plus **filebeat** → **Elasticsearch** → **Kibana**. The concept is identical: *collect everywhere, search in one place, alert on patterns*.

8.2 Centralise the logs

Point every host's logs at the sensor so the dashboard sees the whole network:

```
# /etc/rsyslog.d/90-forward.conf (on each monitored host)
*. * @10.10.30.50:514      # @@ = TCP; send everything to the sensor
```

```
sudo systemctl restart rsyslog
```

The sources you are aggregating:

Source	Log	Tells you
Firewall	dropped-packet syslog	blocked / scanned traffic
SSH hosts	/var/log/auth.log	login successes & failures
Suricata	eve.json	signature alerts (scans, exploits)
Squid	access.log	every web request & denial

Source	Log	Tells you
Nginx	access.log	web attack patterns (404 floods)

8.3 Chain 1 – SSH brute force, blocked by Fail2Ban

◎ OBJECTIVE

Watch a single attack travel through detection and automatic response.

1. **Attack.** From Kali: `hydra -l testuser -P rockyou.txt ssh://10.10.10.20`.
2. **Detect.** On the target, `auth.log` fills with "Failed password" lines; these forward to the dashboard and spike a "failed SSH logins" panel.
3. **Alert.** Suricata's SSH rule fires in `eve.json`; the SIEM raises an alert tagged by source IP.
4. **Respond.** Fail2Ban hits its threshold and bans Kali's IP: `sudo fail2ban-client status sshd` shows it banned.
5. **Confirm.** Further attempts from Kali time out. The dashboard shows the attack starting, the alert, and then *silence* after the ban.

VERIFY IT WORKS

You can tell the story from the dashboard alone: spike → alert → ban → traffic stops. That narrative is exactly what a SOC analyst writes up in an incident ticket.

8.4 Chain 2 – Kali attack → firewall log → IDS alert → dashboard



1. **Attack.** From Kali: `nmap -sS -p- 10.10.30.10` (a full port sweep of the DMZ server).
2. **Firewall.** Ports that are closed/blocked produce DROP entries in the firewall's syslog – a fan-out of one source IP hitting many destination ports.
3. **IDS.** Suricata recognises the scan pattern and writes a port-scan alert to `eve.json`.
4. **Dashboard.** Both signals land in the SIEM. You build a saved search/visualisation: "one source IP, many destination ports, in a short window" – the canonical scan signature.

VERIFY IT WORKS

In the dashboard, filter by Kali's IP and confirm you see *correlated* evidence from two independent sources (firewall + IDS) describing the same event. Correlation across sources is the core skill of detection engineering – you just did it.

THEORY

Why correlate at all? Any single log can mislead. A firewall DROP alone might be benign; an IDS alert alone might be a false positive. *Together*, pointing at the same source IP in the same window, they form high-confidence evidence. Real SOCs live in this correlation layer – this lab teaches it on hardware you fully control.

8.5 Detection engineering: write your own rules

THEORY

Watching default alerts pop onto a dashboard is monitoring, not detection *engineering*. The real skill is writing detections for the threats *you* care about, tuning out false positives, and mapping each alert to a known technique. In Wazuh the workflow is two steps people often miss: a **decoder** first parses a raw log line into fields, then a **rule** matches on those fields – including frequency-based correlation across many events.

OBJECTIVE

A custom Wazuh rule that fires a Critical (level 12) alert when one source IP causes more than 10 failed proxy/web authentications within 30 seconds, tagged with its MITRE ATT&CK technique.

8.5.1 Decode the log, then write the rule

```
<!-- /var/ossec/etc/decoders/local_decoder.xml : extract IP + status -->
<decoder name="nginx-auth-fail">
  <program_name>nginx</program_name>
  <regex>client (\d+.\d+.\d+.\d+).*401</regex>
  <order>srcip</order>
</decoder>
```

```
<!-- /var/ossec/etc/rules/local_rules.xml -->
<group name="web,authentication,attack,">

  <!-- base rule: a single failed web auth -->
  <rule id="100100" level="5">
    <decoded_as>nginx-auth-fail</decoded_as>
    <description>Failed web/proxy authentication</description>
  </rule>

  <!-- correlation rule: >10 failures from one IP in 30s = brute force -->
  <rule id="100101" level="12" frequency="10" timeframe="30">
    <if_matched_sid>100100</if_matched_sid>
    <same_source_ip />
    <description>CRITICAL: web auth brute force from $(srcip)</description>
    <mitre>
      <id>T1110</id>    <!-- Brute Force -->
    </mitre>
  </rule>
</group>
```

```
sudo systemctl restart wazuh-manager
# test a rule against a sample log line WITHOUT waiting for a real attack:
sudo /var/ossec/bin/wazuh-logtest
```

VERIFY IT WORKS

Replay the brute force from Kali against the proxy/web auth. The dashboard shows a burst of level-5 events, then a single level-12 *correlated* alert naming the source IP and tagged T1110. You wrote a detection, not just watched one.

8.5.2 Make detections portable: Sigma**THEORY**

A rule written in Wazuh's syntax only runs in Wazuh. **Sigma** is a vendor-neutral detection format that converts to Wazuh, Splunk, Elastic, and more – so the same logic follows you between jobs and tools. Writing your detections as Sigma first, then converting, is the portable, resume-worthy version of this skill.

```
# author a Sigma rule (YAML), then convert it for your backend
pip install sigma-cli
sigma convert -t elasticsearch detection.yml # or another supported target
```

Where this goes next: every attack you ran in this lab – port scans, brute force, web injection, directory enumeration – has a MITRE technique ID. Re-walking your detections and asking "which ATT&CK technique does this catch, and what am I still blind to?" is exactly how a detection engineer audits coverage.

MODULE 9

Cloud Security (AWS)

Everything you built on-premises has a cloud equivalent. This module rebuilds the same secure architecture on AWS so you can map each concept across worlds. Most of this fits comfortably in the AWS Free Tier; **always tear resources down afterwards** to avoid charges.

THEORY

The mapping that makes cloud click: a **VPC** is your private network (like the whole lab); **subnets** are your segments (LAN/DMZ); **Security Groups** are stateful per-instance firewalls; **NACLs** are stateless per-subnet firewalls; a **NAT Gateway** is your outbound NAT; a **bastion host** replaces "SSH from the LAN"; **IAM** is least privilege for the cloud control plane; **CloudTrail/CloudWatch** are your central logs and dashboard.

9.1 VPC and subnets

OBJECTIVE

A VPC with a public subnet (for the load balancer / bastion) and a private subnet (for app servers), mirroring DMZ vs trusted LAN.

```
aws ec2 create-vpc --cidr-block 10.20.0.0/16
aws ec2 create-subnet --vpc-id vpc-xxxx --cidr-block 10.20.1.0/24 # public
aws ec2 create-subnet --vpc-id vpc-xxxx --cidr-block 10.20.2.0/24 # private
# public subnet gets an internet gateway + route; private does not
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-xxxx --internet-gateway-id igw-xxxx
```

9.2 Security Groups (stateful) vs NACLs (stateless)

THEORY

A **Security Group** is exactly the stateful firewall from Module 3, attached to an instance: allow rules only, return traffic automatic. A **NACL** is the stateless subnet-level firewall: it needs both inbound *and* outbound rules and is evaluated by rule number. Use SGs for normal control; use NACLs as a coarse subnet guardrail (e.g. block a hostile IP range for the whole subnet).

```
# SG: web server allows 443 from anywhere, SSH only from the bastion's SG
aws ec2 create-security-group --group-name web-sg --description "web" --vpc-id vpc-xxxx
aws ec2 authorize-security-group-ingress --group-id sg-web \
  --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id sg-web \
  --protocol tcp --port 22 --source-group sg-bastion # SSH only from bastion
```

VERIFY IT WORKS

This is your "block SSH from the internet" lesson, cloud edition: port 22 references the bastion's SG, never 0.0.0.0/0. Confirm in the console that no SG anywhere allows 22 from the internet.

9.3 NAT Gateway

THEORY

Private-subnet instances have no public IP, so they cannot reach the internet directly (good – they are hidden). A **NAT Gateway** in the public subnet lets them make *outbound* connections (updates, APIs) without being reachable *inbound*. Same SNAT idea as your firewall's MASQUERADE.

```
aws ec2 create-nat-gateway --subnet-id subnet-public --allocation-id eipalloc-xxxx
# then add a route in the PRIVATE subnet: 0.0.0.0/0 -> the NAT gateway
```

9.4 Bastion host

◎ OBJECTIVE

The only SSH entry point: a small hardened instance in the public subnet; admins reach private instances *through* it.

```
# connect to a private instance by jumping through the bastion (one command)
ssh -J ec2-user@<bastion-public-ip> ec2-user@10.20.2.15
```

This is the cloud version of Module 5's "VPN instead of exposed SSH". The bastion is the single audited choke point; harden it exactly as in Module 7 (keys only, no root, Fail2Ban). Modern alternative: AWS Systems Manager Session Manager, which removes the need for an open SSH port entirely.

9.5 Load balancer + WAF

◎ OBJECTIVE

An Application Load Balancer fronts the app (your reverse proxy, managed), with a WAF filtering malicious requests – the cloud answer to Module 6's web scans.

THEORY

The **ALB** terminates TLS and spreads traffic across healthy instances (the reverse-proxy role). **AWS WAF** attaches to the ALB and blocks attack patterns – SQL injection, cross-site scripting, bad bots, and rate-limit floods – before they reach your app. Remember the Nikto/Gobuster scans? A WAF rate-based rule blocks exactly that 404 flood.

```
# attach a managed rule set + a rate limit to the WAF web ACL
aws wafv2 create-web-acl --name lab-waf --scope REGIONAL \
  --default-action Allow={} \
  --rules file://managed-rules.json # includes AWS Managed Common + rate-based
# associate the web ACL with the ALB
aws wafv2 associate-web-acl --web-acl-arn <arn> --resource-arn <alb-arn>
```

9.6 IAM least privilege

THEORY

IAM controls who can do what to your cloud account itself – the most consequential security layer, because IAM mistakes hand attackers the keys to everything. Apply the same least-privilege rule from Module 7: grant the minimum actions on the minimum resources, prefer roles over long-lived keys, and enable MFA on humans.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::lab-app-bucket/*"
  }]
}
```

WATCH OUT

Never attach **AdministratorAccess** to application roles, and never commit access keys to code. The overwhelmingly common cloud breach is a leaked over-privileged key. Scope tightly; rotate; use roles.

9.7 Logging & monitoring: CloudTrail + CloudWatch

THEORY

CloudTrail records every API call made in your account (who did what, when) – the audit trail. **CloudWatch** collects metrics and logs and raises alarms. Together they are the cloud version of your SIEM dashboard from Module 8.

```
aws cloudtrail create-trail --name lab-trail --s3-bucket-name lab-trail-logs
aws cloudtrail start-logging --name lab-trail
# alarm on suspicious activity, e.g. root account usage
aws cloudwatch put-metric-alarm --alarm-name root-login \
  --metric-name RootAccountUsage --threshold 1 --evaluation-periods 1 ...
```

VERIFY IT WORKS

Perform an action (launch an instance), then find it in CloudTrail's event history filtered by your user – the same "who touched this" investigation you ran with logs in Module 8, now

for the cloud control plane.

WATCH OUT

Tear down to avoid charges: delete the NAT Gateway, ALB, and any running instances when you finish a session – these bill by the hour even when idle. `terraform destroy` or a teardown script is the disciplined habit.

MODULE 10

PKI & Certificate Authority

Welcome to the advanced track. The modules from here on lift the lab from a secure network into modern, cloud-native security engineering: your own PKI, Linux internals, containers and Kubernetes, infrastructure-as-code, CI/CD supply-chain security, deeper AWS, detection engineering at depth, and a purple-team capstone that chains everything together. Each still follows Theory → Objective → Verify → Red Team.

You have used certificates several times already – the Squid bump CA in Module 4, OpenVPN's CA in Module 5, TLS on the reverse proxy. This module formalises *how trust actually works* and has you run a real internal Certificate Authority with a root, an intermediate, issued certificates, mutual TLS, and revocation.

THEORY

What a PKI is. Public-Key Infrastructure is the system that lets one party trust another's public key. A **Certificate Authority** signs certificates that bind a public key to an identity (a hostname, a user, a service). Anything trusting the CA then trusts everything it signs. Real PKIs use a two-tier chain: an offline **root CA** signs one or more **intermediate CAs**, and only intermediates issue end-entity ("leaf") certs – so if an intermediate is compromised you revoke it without burning the root. Trust flows root → intermediate → leaf; this is a *chain of trust*.

10.1 Build a two-tier internal CA

OBJECTIVE

A root CA (kept offline), an intermediate CA that does the day-to-day signing, and a server certificate for `web.lab.local` that chains cleanly to the root.

```
# 1) Root CA: a long-lived, self-signed key you then lock away
openssl genrsa -aes256 -out root.key 4096
openssl req -x509 -new -key root.key -days 7300 -sha256 \
  -out root.crt -subj "/CN=Lab Root CA"

# 2) Intermediate CA: signed BY the root, used for daily issuance
openssl genrsa -out intermediate.key 4096
openssl req -new -key intermediate.key -out intermediate.csr \
  -subj "/CN=Lab Intermediate CA"
openssl x509 -req -in intermediate.csr -CA root.crt -CAkey root.key \
  -CAcreateserial -days 3650 -sha256 \
  -extfile <(printf "basicConstraints=critical,CA:TRUE,pathlen:0\nkeyUsage=critical,\nkeyCertSign,cRLSign") \
  -out intermediate.crt

# 3) Leaf cert for a server, signed by the intermediate
openssl genrsa -out web.key 2048
openssl req -new -key web.key -out web.csr -subj "/CN=web.lab.local"
```

```
openssl x509 -req -in web.csr -CA intermediate.crt -CAkey intermediate.key \
-CACreateserial -days 825 -sha256 \
-extfile <(printf "subjectAltName=DNS:web.lab.local\nkeyUsage=digitalSignature,\
keyEncipherment\nextendedKeyUsage=serverAuth") \
-out web.crt

# build the chain the server presents (leaf + intermediate)
cat web.crt intermediate.crt > web-fullchain.crt
```

VERIFY IT WORKS

Verify the leaf chains to the root through the intermediate:

```
openssl verify -CAfile root.crt -untrusted intermediate.crt web.crt
# web.crt: OK
openssl x509 -in web.crt -noout -text | grep -A1 "Subject Alternative"
```

Distribute `root.crt` to your hosts' trust stores (Module 7 showed `update-ca-certificates`); now every internal TLS service signed by your intermediate is trusted with no browser warnings.

10.2 Mutual TLS (mTLS): both sides prove identity

THEORY

Ordinary TLS authenticates the *server* to the client. **Mutual TLS** also authenticates the *client* to the server: the client presents its own certificate, signed by the same CA. This is how zero-trust service-to-service auth works – a service only talks to peers holding a valid cert from the trusted CA. You will see exactly this inside Kubernetes service meshes later.

```
# issue a CLIENT cert (extendedKeyUsage = clientAuth) from the intermediate
openssl genrsa -out client.key 2048
openssl req -new -key client.key -out client.csr -subj "/CN=svc-client"
openssl x509 -req -in client.csr -CA intermediate.crt -CAkey intermediate.key \
-CACreateserial -days 825 -sha256 \
-extfile <(printf "extendedKeyUsage=clientAuth") -out client.crt
```

```
# nginx: require a client cert signed by our CA
server {
    listen 443 ssl;
    ssl_certificate      web-fullchain.crt;
    ssl_certificate_key  web.key;
    ssl_client_certificate /etc/nginx/ca/root-and-intermediate.crt;
    ssl_verify_client on;      # <- reject anyone without a valid client cert
}
```

VERIFY IT WORKS

A request *with* the client cert succeeds; without it, the server refuses at the TLS layer:

```
curl --cacert root.crt --cert client.crt --key client.key https://web.lab.local
# 200
curl --cacert root.crt https://web.lab.local # 400 No required SSL certificate
```

10.3 Automated issuance & revocation

THEORY

Manual `openssl` is great for learning but does not scale. In production, leaf certs are issued automatically via **ACME** (the protocol behind Let's Encrypt) and rotated before they expire, because short-lived certs limit the damage of a leak. And when a key *is* compromised you need **revocation**: a CRL (certificate revocation list) or OCSP tells relying parties "do not trust this cert anymore."

```
# stand up an internal ACME CA so services auto-enrol (step-ca is ideal for a lab)
step ca init --name "Lab CA" --dns ca.lab.local --address :443 --provisioner admin
step-ca $(step path)/config/ca.json &
# a host requests + auto-renews a cert with one agent -- no manual openssl
step ca certificate web.lab.local web.crt web.key

# revoke a leaked cert, publish the CRL
openssl ca -revoke compromised.crt -keyfile intermediate.key -cert intermediate.crt
openssl ca -gencrl -keyfile intermediate.key -cert intermediate.crt -out lab.crl
```

Through-line: this internal CA now backs everything – the proxy bump cert, VPN certs, internal HTTPS, mTLS between services, and the certificates Kubernetes and your CI runners will need. One trust anchor, many uses.

MODULE 11

Linux Internals & nftables

To secure Linux deeply – and to understand containers in the next module – you need the kernel primitives underneath: namespaces, cgroups, capabilities, seccomp, and Mandatory Access Control. You also need **nftables**, the modern replacement for the iptables you used throughout the firewall module.

11.1 nftables: the modern packet filter

THEORY

iptables (used in Module 3) is being superseded by **nftables**: one unified tool for IPv4/IPv6/ARP, a cleaner syntax, atomic rule replacement (no half-applied rulesets), and native *sets* and *maps* that make large allow/deny lists fast and readable. The concepts are identical – tables, chains, hooks, stateful matching – but the expression is far better. On modern distros, **iptables** commands are often just a compatibility shim over nftables anyway.

OBJECTIVE

Re-express the firewall's default-deny stateful policy from Module 3 in native nftables, using a set for allowed admin IPs.

```
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
    # a named set: edit the allow-list without touching rules
    set admins { type ipv4_addr; elements = { 10.10.10.10, 10.10.99.10 } }

    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept          # stateful return traffic
        iif lo accept
        tcp dport 22 ip saddr @admins accept         # SSH only from the set
        ip protocol icmp accept
    }
    chain forward {
        type filter hook forward priority 0; policy drop;
        ct state established,related accept
        iifname "lan" oifname "wan" accept           # LAN may initiate outbound
        iifname "dmz" oifname "lan" drop             # DMZ must NOT reach the LAN
    }
}
```

```
sudo nft -f /etc/nftables.conf      # apply atomically
sudo nft list ruleset                # read back the live ruleset
sudo nft add element inet filter admins { 10.10.10.11 } # grow the set, no rule
edit
```

VERIFY IT WORKS

`sudo nft list set inet filter admins` shows the live allow-list; SSH works only from a member IP; the DMZ-to-LAN drop still holds. Same policy as Module 3, expressed in the tool you will actually meet on new systems.

11.2 Namespaces & cgroups: how isolation works

THEORY

Namespaces give a process its own private view of a resource – its own network stack (**net**), process tree (**pid**), mounts, users, and so on. **cgroups** (control groups) limit and account for what a process may consume (CPU, memory, PIDs). Together they *are* containers: a "container" is just a normal process running in its own namespaces with cgroup limits and a restricted filesystem. Understanding this demystifies the entire next module and explains why container escapes are really namespace/capability failures.

```
# create an isolated network namespace by hand -- this is what a container does
sudo ip netns add sandbox
sudo ip netns exec sandbox ip addr          # its own, empty network stack
# run a shell in fresh namespaces with unshare (no Docker involved):
sudo unshare --pid --net --mount --uts --fork bash
hostname isolated && ps aux                 # its own hostname + process tree
# inspect which namespaces any process is in:
ls -l /proc/$$/ns
```

11.3 Capabilities: break up root

THEORY

Historically a process was either root (all-powerful) or not. **Capabilities** split root's powers into ~40 distinct privileges (**CAP_NET_BIND_SERVICE** to bind low ports, **CAP_NET_ADMIN** to configure networking, etc.). Least privilege means granting a service *only* the capabilities it needs and dropping the rest – so a compromised web server that needs port 80 cannot also load kernel modules.

```
# let a binary bind port 80 WITHOUT running as root
sudo setcap 'cap_net_bind_service=+ep' /usr/bin/myserver
getcap /usr/bin/myserver
# see a running process's capabilities
grep Cap /proc/$(pgrep -n nginx)/status
```

11.4 seccomp & Mandatory Access Control

THEORY

seccomp filters which *syscalls* a process may make – shrinking the kernel attack surface to only what the app needs. **MAC** systems (**AppArmor** on Debian/Ubuntu, **SELinux** on RHEL) confine a process to a policy regardless of file permissions, so even a root-owned compromised service can only touch what its profile allows. These two are what make container and host hardening actually hold.

```
# AppArmor: see and enforce profiles
sudo aa-status                # which profiles are loaded / enforcing
sudo aa-enforce /etc/apparmor.d/usr.sbin.nginx
# auditd: record security-relevant events (who changed what, when)
sudo apt install auditd -y
sudo auditctl -w /etc/passwd -p wa -k passwd_changes # watch a sensitive file
sudo ausearch -k passwd_changes                     # review the audit trail
```

RED TEAM – ATTACK

See least-privilege bite. Run a process with all capabilities dropped and watch a privileged action fail:

```
# this container/process cannot change network config even as "root"
sudo unshare --net bash -c 'ip link add dummy0 type dummy' # works (has
CAP_NET_ADMIN)
capsh --drop=cap_net_admin -- -c 'ip link add dummy0 type dummy' # Operation
not permitted
```

That single denied syscall is the whole philosophy: take away powers nothing legitimate needs, and many exploits simply cannot run.

Systemd hardening tie-in: the same primitives are exposed declaratively in unit files – `CapabilityBoundingSet`, `SystemCallFilter`, `ProtectSystem=strict`, `NoNewPrivileges=yes`. Hardening a service is largely choosing the right subset of these.

11.5 PAM, systemd & journald: the auth and service plumbing

THEORY

Three subsystems decide who logs in, what runs, and what gets recorded. **PAM** (Pluggable Authentication Modules) is the stack every login flows through – it is where you enforce password policy, lockouts, and MFA (the `pam_google_authenticator` from Module 7 plugs in here). **systemd** starts and supervises services, and its unit directives are a hardening surface in their own right. **journald** is the structured log everything writes to – your first stop in any investigation and a feed into the SIEM.

```
# PAM: see the stack a login traverses, and the lockout module that fights brute
force
cat /etc/pam.d/sshd
grep -r pam_faillock /etc/pam.d/ # lock an account after N failures (enterprise
default)
```

```
# systemd: turn the hardening directives into a real, sandboxed service
sudo systemctl edit myapp          # drop-in override:
# [Service]
#   NoNewPrivileges=yes
#   ProtectSystem=strict
#   ProtectHome=yes
#   PrivateTmp=yes
#   CapabilityBoundingSet=CAP_NET_BIND_SERVICE
systemd-analyze security myapp.service # GRADES your unit's exposure -- aim to
lower it

# journald: query structured logs by service, priority, and time
journalctl -u ssh --since "1 hour ago" -p err
journalctl _UID=1000 -o json          # machine-readable, ready to ship to the SIEM
```

VERIFY IT WORKS

`systemd-analyze security myapp.service` shows the exposure score *dropping* after you add the sandboxing directives – a concrete, measurable hardening you can point to. `pam_faillock` locks a test account after the configured failures, the host-level twin of Fail2Ban.

SSH internals worth knowing

Beyond Module 7's config, understand the moving parts: the host key (proves the *server*'s identity – "host key changed" warnings mean exactly that), the `known_hosts` trust-on-first-use model, the authentication order (publickey before password), and the agent (`ssh-add`) plus safe agent forwarding versus the safer `ProxyJump` you used for the bastion. `ssh -vvv` narrates the entire negotiation when something breaks.

MODULE 12

Windows Enterprise Security & Detection

The lab has been Linux-heavy, but the enterprises you will defend are not: the majority of corporate endpoints, and therefore the majority of real detection work, are Windows. You stood up Active Directory in Module 1 and ran AD attacks; this module gives you the *Windows-side telemetry and defences* to detect them, which is the single biggest employability gap in an otherwise Linux lab.

THEORY

Why Windows realism matters. Most SOC alerts, most EDR detections, and most ATT&CK techniques in the wild target Windows. A security engineer who cannot read a Windows Security event log or deploy Sysmon is missing the platform where the work actually happens. The good news: the *concepts* are identical to what you already know – centralised logs, baseline, detect anomalies – only the telemetry source changes.

12.1 Windows logging: the events that matter

OBJECTIVE

A Windows endpoint (or your domain controller from Module 1) generating the security events a defender relies on, and you able to read them.

Learn these core Security-log Event IDs cold – they are the vocabulary of Windows detection:

Event ID	Meaning (why a defender cares)
4624 / 4625	Successful / failed logon (logon type reveals interactive vs network vs RDP)
4672	Special privileges assigned – an admin/SYSTEM logon
4688	A new process was created (with command line, if auditing is on)
4768 / 4769	Kerberos TGT / service-ticket request (4769 with RC4 = Kerberoasting signal)
4720 / 4732	User account created / added to a privileged group
7045	A new service was installed (classic persistence + lateral movement)

```
# read the Security log from PowerShell (run as admin)
Get-WinEvent -LogName Security -MaxEvents 50 |
  Where-Object Id -in 4624,4625,4672,4688
# enable command-line capture in 4688 so you SEE what ran (via GPO or:)
# Audit Process Creation = Success + "Include command line in process events"
```


12.2 Sysmon: the defender's microscope

THEORY

The built-in logs are coarse. **Sysmon** (System Monitor, from Sysinternals) adds rich, high-fidelity telemetry: every process creation with its parent and hashes, network connections, image/DLL loads, registry changes, and more. A good Sysmon config is the foundation of nearly every Windows detection – it is to Windows what auditd + Falco are to Linux.

```
# install Sysmon with a community baseline config (SwiftOnSecurity's is the standard)
sysmon64.exe -accepteula -i sysmonconfig.xml
# Sysmon writes here; Event ID 1 = process create, 3 = network, 7 = image load
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 20
```

VERIFY IT WORKS

Run `whoami` or open a browser, then find the corresponding Sysmon Event ID 1 showing the full command line, parent process, and SHA-256 hash. You are now seeing process lineage – the data behind "this Word document spawned PowerShell," the canonical malware pattern.

12.3 Ship Windows logs to your SIEM

THEORY

Isolated Windows logs are useless at scale. **Windows Event Forwarding** (WEF) or an agent (Winlogbeat / the Wazuh agent from Module 8) ships Security + Sysmon events to the same SIEM that already holds your Linux, firewall, and proxy logs – one correlation point for the whole estate.

```
# install the Wazuh agent on Windows and point it at the manager from Module 9
# it forwards Security + Sysmon channels automatically once configured
# in ossec.conf on the endpoint:
# <localfile><location>Microsoft-Windows-Sysmon/Operational</location>
# <log_format>eventchannel</log_format></localfile>
```

12.4 Detect the AD attacks you already ran

RED TEAM – ATTACK

Re-run the Module 1 directory attacks against your DC and watch the Windows-side signatures appear:

```
# Kerberoasting -> Event 4769 with Ticket Encryption Type 0x17 (RC4) for an SPN
# DCSync (replicating secrets) -> Event 4662 with the DS-Replication GUID
# Pass-the-Hash / lateral logon -> 4624 Logon Type 3 with NTLM from an odd
source
GetUserSPNs.py lab.local/jdoe -dc-ip 10.10.10.5 -request # from Kali (M.
identity)
```

VERIFY IT WORKS

After the Kerberoast, a query for Event 4769 where the encryption type is RC4 returns your attack – that single condition is a high-fidelity Kerberoasting detection, and you can now write it as a Sigma rule (Module 8) and map it to ATT&CK T1558.003 in your coverage matrix (Module 18).

12.5 Windows hardening enterprises actually use

THEORY

Detection is half; reduce the attack surface too. The standard enterprise controls: **LAPS** (randomises and rotates each machine's local-admin password, killing lateral movement via a shared local password), **GPO**-driven baselines (disable SMBv1, enforce SMB signing, restrict NTLM), **Credential Guard** (protects credentials in memory from theft), and **AppLocker** / **WDAC** (application allow-listing). Tiered administration – never use a Domain Admin account to log into a workstation – is the structural defence against the BloodHound-style paths you mapped earlier.

WATCH OUT

The classic mistake that BloodHound exploits: a Domain Admin logs into a normal workstation, leaving their credentials in memory for any attacker who later compromises that machine. *Tiering* (admin accounts only ever touch their tier) is the fix, and it is an architecture decision, not a tool. Detect violations by alerting on Tier-0 account logons (4624) to non-Tier-0 hosts.

Resume / interview value: "I deployed Sysmon, forwarded Windows + Sysmon telemetry to a SIEM, and wrote detections for Kerberoasting (4769/RC4), DCSync (4662), and Pass-the-Hash (4624/type 3), mapped to ATT&CK" is a sentence that lands a junior detection-engineering or SOC role. It is the exact daily work.

MODULE 13

Container & Kubernetes Security

Containers are now how almost everything ships, so they are now where almost everything gets attacked. Building on the namespaces, cgroups, and capabilities from Module 11, this module hardens container images and runtime, then secures a Kubernetes cluster: RBAC, network policies, pod security, secrets, admission control, and runtime detection.

13.1 Docker: harden the image and the runtime

THEORY

A container is only as safe as its image and how you run it. The two biggest wins: build *minimal* images that run as a *non-root* user, and run them with dropped capabilities, a read-only filesystem, and **no-new-privileges**. Most "container escapes" start with a container running as root with extra capabilities it never needed.

OBJECTIVE

A hardened image built non-root from a minimal base, scanned for known CVEs, and run with a locked-down runtime profile.

```
# Dockerfile: minimal base, non-root, multi-stage to drop build tooling
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app

FROM gcr.io/distroless/static:nonroot      # tiny, no shell, runs as nonroot
COPY --from=build /app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

```
# scan the image for known vulnerabilities BEFORE you ship it
trivy image myapp:latest --severity HIGH,CRITICAL
# run with a hardened runtime profile
docker run --read-only --cap-drop ALL --security-opt no-new-privileges \
  --user 1000:1000 --pids-limit 100 myapp:latest
```

VERIFY IT WORKS

`docker run myapp whoami` returns the non-root user (or fails – distroless has no shell, which is the point). Trivy reports 0 HIGH/CRITICAL once you have patched the base.
`docker inspect` shows `CapDrop: ALL` and `ReadOnlyRootfs: true`.

RED TEAM – ATTACK

Why this matters. A container run with `-privileged` or `-v /:/host` can trivially escape to the host. Demonstrate the danger (on a throwaway VM only):

```
docker run --rm -v /:/host alpine cat /host/etc/shadow # reads the HOST's shadow file!
```

Then prove the hardened run cannot: the read-only, cap-dropped, non-root container has no path to the host. Never use `-privileged` or mount the Docker socket into a container you do not fully trust.

13.2 Kubernetes: a hardened cluster

◎ OBJECTIVE

A single-node cluster (k3s or minikube) where workloads run with least privilege, network policies isolate namespaces, secrets are protected, and a policy engine rejects unsafe pods.

```
# fastest path to a real cluster in the lab
curl -sfL https://get.k3s.io | sh - # k3s: a full, lightweight K8s
kubectl get nodes
```

13.2.1 RBAC: least privilege for the cluster

THEORY

Kubernetes RBAC mirrors the directory RBAC from Module 1: *Roles* grant verbs (get, list, create) on resources, and *RoleBindings* attach them to users or service accounts. The default service account a pod gets is a classic over-privilege; scope it down.

```
# read-only role, bound to one service account in one namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: app, name: pod-reader }
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { namespace: app, name: read-pods }
subjects: [{ kind: ServiceAccount, name: app-sa, namespace: app }]
roleRef: { kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io }
```

13.2.2 Network policies: default-deny inside the cluster

THEORY

By default every pod can talk to every other pod – a flat network, the same anti-pattern you fixed with segmentation in Module 3. A **NetworkPolicy** restores default-deny *inside* the cluster, then you allow only the flows you need.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: app }
spec:
  podSelector: {}           # all pods in the namespace
  policyTypes: ["Ingress", "Egress"] # deny both directions by default
```

13.2.3 Pod Security, secrets, and admission control

```
# enforce the "restricted" Pod Security Standard on a namespace
kubectl label ns app pod-security.kubernetes.io/enforce=restricted

# a pod's securityContext: non-root, no privilege escalation, read-only fs
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: { drop: ["ALL"] }
```

WATCH OUT

Kubernetes Secrets are only base64, not encrypted, by default. Anyone with read access to the secret (or to etcd) can decode them. Enable *encryption at rest* for etcd, restrict secret access with RBAC, and for real use pull from an external store (the Vault integration in Module 14). Treat a cluster admin token like a domain admin password from Module 1.

```
# admission control: reject non-compliant pods cluster-wide with Kyverno
kubectl apply -f https://github.com/kyverno/kyverno/releases/latest/download/install.
  yaml
# example policy: block any pod that runs as root or is privileged
kubectl apply -f disallow-privileged.yaml
```

13.2.4 Runtime detection with Falco

THEORY

RBAC and policies are *preventive*; you also need *detection* of what happens at runtime. **Falco** watches kernel syscalls and alerts on suspicious container behaviour – a shell spawned in a container, a write to a sensitive path, an unexpected outbound connection. It is the in-cluster equivalent of the IDS from Module 6.

```
helm install falco falcosecurity/falco -n falco --create-namespace
```

RED TEAM – ATTACK

Trigger a detection: exec a shell into a running pod – a hallmark of an attacker who has gained a foothold.

```
kubectrl exec -it <pod> -- /bin/sh
kubectrl logs -n falco -l app.kubernetes.io/name=falco | grep "Terminal shell in container"
```

VERIFY IT WORKS

A pod manifest that runs as root or requests **privileged: true** is *rejected at admission* by Kyverno; the default-deny NetworkPolicy blocks pod-to-pod traffic you did not explicitly allow; and Falco alerts the moment a shell opens in a container. Prevention plus detection, both in place.

Scan everything, continuously: run **trivy k8s cluster** and a CIS benchmark tool like **kube-bench** to grade the cluster against best practice – this becomes a CI gate in Module 15.

MODULE 14

Infrastructure as Code: Terraform, Ansible & Secrets

You used Ansible in Module 7 to *configure* existing hosts. The missing half is *provisioning* the infrastructure itself as code, scanning that code for misconfigurations before it ships, and handling secrets without ever committing them. This module makes the whole environment reproducible and auditable – "security as code."

THEORY

Two tools, two jobs. **Terraform** *provisions* infrastructure (VPCs, instances, security groups) declaratively – you describe the desired state and it creates/changes/destroys to match. **Ansible** *configures* what exists (packages, files, services). The standard pattern is Terraform builds the boxes, then hands off to Ansible to harden them. Both are declarative and idempotent, which is what makes infrastructure reviewable in pull requests like application code.

14.1 Provision with Terraform

◎ OBJECTIVE

The AWS network from Module 9 – VPC, subnets, a least-privilege security group – defined in Terraform so it can be created and destroyed reproducibly.

```
# main.tf -- a security group that mirrors the Module-9 policy, as code
resource "aws_security_group" "web" {
  name     = "web-sg"
  vpc_id   = aws_vpc.lab.id

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  # HTTPS from anywhere

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    security_groups = [aws_security_group.bastion.id]
  }
  # SSH ONLY from the bastion SG -- never 0.0.0.0/0

  egress { from_port = 0; to_port = 0; protocol = "-1"; cidr_blocks = ["0.0.0.0/0"] }
}
```

```
terraform init
terraform plan      # preview EXACTLY what will change before it happens
terraform apply
terraform destroy   # tear it all down cleanly (the discipline from Module 9)
```

14.2 Scan the code before it ships

THEORY

The huge security advantage of IaC is that misconfigurations become *reviewable and scannable* before anything is built. Tools like `tfsec`/`trivy config` and `checkov` catch "this S3 bucket is public", "this SG allows 22 from the world", or "this disk is unencrypted" at the pull-request stage – shifting security *left*, before deployment.

```
trivy config .                # scans Terraform/Dockerfiles/K8s manifests for
    misconfig
checkov -d .                  # policy-as-code checks with hundreds of built-in
    rules
```

VERIFY IT WORKS

Deliberately add an ingress rule allowing 22 from 0.0.0.0/0, re-run `trivy config .`, and watch it flag the open-SSH finding with a severity and a fix. Remove it, re-scan, finding gone. You just caught a firewall mistake *before* it ever existed in the cloud.

14.3 State files are secret – treat them so

WATCH OUT

Terraform **state** (`terraform.tfstate`) records your real infrastructure and often contains *secrets in plaintext* (generated passwords, keys). Never commit it to git. Store it in an encrypted, access-controlled remote backend (e.g. an S3 bucket with encryption + a DynamoDB lock table), and restrict who can read it exactly as you would a credentials file.

14.4 Secrets management with Vault

THEORY

Secrets (DB passwords, API keys, certificates) must never live in code, container images, or Terraform variables checked into git. A secrets manager like **HashiCorp Vault** stores them encrypted, hands them out only to authenticated callers, audits every access, and can issue *dynamic, short-lived* credentials that expire automatically – so a leaked secret is useless minutes later. This is the single most common gap in real environments.

🎯 OBJECTIVE

A Vault server that stores a database credential and serves it to an authenticated client, with every access logged.

```
vault server -dev &                # dev mode for the lab only
export VAULT_ADDR=http://127.0.0.1:8200
# store and read a static secret
vault kv put secret/db password="s3cr3t"
vault kv get -field=password secret/db
# dynamic secrets: Vault generates a NEW db user on demand that auto-expires
vault secrets enable database
```



```
vault read database/creds/app-role      # returns a temporary username/password
```

VERIFY IT WORKS

The application reads its database password from Vault at startup instead of from a config file, and `vault audit list` shows a logged entry for every secret access. Grep your repository: there are *no* plaintext credentials anywhere in code or state. That is the goal.

Ties it together: Vault issues the dynamic credentials your CI pipeline (Module 15) and Kubernetes pods (Module 13) need, and can act as a PKI engine that issues short-lived certs from the CA you built in Module 10. One secrets backbone for the whole lab.

MODULE 15

CI/CD & Supply-Chain Security

Modern attacks increasingly hit the *pipeline*, not the running server: poison a dependency, steal a CI token, or tamper with a build, and you compromise everything that pipeline deploys. This module builds a secure CI/CD pipeline that gates code on security checks and protects the supply chain end to end.

THEORY

The shift-left idea. Catching a vulnerability in a pull request costs minutes; catching it in production costs an incident. A secure pipeline runs automated security gates on every commit – secret scanning, dependency analysis, static and dynamic testing, IaC and container scanning – and *fails the build* when they trip. The pipeline itself is also an attack surface: its credentials, its runners, and the artifacts it produces all need protecting.

15.1 The security gates in a pipeline

Gate	Tool (example)	Catches
Secret scanning	gitleaks, trufflehog	credentials committed to the repo
SCA (dependencies)	trivy, gype, <code>npm audit</code>	known CVEs in third-party libraries
SAST (your code)	semgrep, CodeQL	injection, unsafe code patterns
IaC scanning	trivy config, checkov	misconfigured cloud/K8s/Terraform
Container scanning	trivy image	vulnerable base images
DAST (running app)	OWASP ZAP	runtime web vulnerabilities

15.2 Build a gated pipeline

🎯 OBJECTIVE

A GitHub Actions (or GitLab CI) pipeline that runs secret, dependency, SAST, IaC, and image scans on every push and blocks the merge if any HIGH/CRITICAL issue is found.

```
# .github/workflows/security.yml
name: security-gates
on: [push, pull_request]
permissions:
  contents: read          # least privilege: the token can only READ the repo
jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v4
- name: Secret scan
  run: gitleaks detect --no-banner
- name: Dependency + IaC + image scan
  run: |
    trivy fs --severity HIGH,CRITICAL --exit-code 1 .
    trivy config --exit-code 1 .
- name: Static analysis
  run: semgrep ci

```

VERIFY IT WORKS

Open a pull request that adds a hard-coded API key and a vulnerable dependency. The pipeline fails: gitleaks flags the key, trivy flags the CVE, and the merge is blocked. Remove them and the pipeline goes green. Security is now enforced by the pipeline, not by hoping someone remembers.

15.3 Protect the supply chain

THEORY

Supply-chain security answers: can you trust what you build and ship? Three pillars: a **SBOM** (software bill of materials) lists every component so you can answer "are we affected by CVE-X?" in seconds; **artifact signing** (Sigstore/cosign) proves an image was built by your pipeline and not swapped; and **provenance/SLSA** attestations record *how* an artifact was built. Without these, a tampered dependency or image is invisible.

```

# generate an SBOM for an image, then scan the SBOM against CVE feeds
syft myapp:latest -o spdx-json > sbom.json
grype sbom:sbom.json
# sign the image so consumers can verify its origin
cosign sign myapp:latest
cosign verify myapp:latest           # fails if the image was tampered/unsigned

```

15.4 Harden the pipeline itself

WATCH OUT

The CI runner often holds the keys to your whole cloud. Real-world rules: give workflow tokens the *minimum* permissions (**permissions:** **contents:** **read**); use short-lived **OIDC** federation to the cloud instead of long-lived stored secrets; pin third-party actions to a commit SHA (not a mutable tag) so a hijacked action cannot inject code; and pull pipeline secrets from Vault (Module 14) rather than storing them in the CI system.

RED TEAM – ATTACK

Think like a pipeline attacker. The classic moves: a malicious dependency that runs code at install time; a "pull_request_target" workflow tricked into running attacker code with secrets; or a leaked CI token used to push a backdoored image. Your gates (secret scan, SCA, pinned actions, signed artifacts, least-privilege tokens) each close one of these doors – map every gate to the attack it stops.

VERIFY IT WORKS

`cosign verify` succeeds for an image your pipeline built and *fails* for an unsigned or modified one; the workflow token cannot write to the repo or assume broad cloud roles; and querying your SBOM instantly tells you whether a newly announced CVE affects you. The supply chain is now verifiable, not assumed.

MODULE 16

Advanced AWS Security

Module 9 built the network primitives. This module adds the services a real AWS security engineer lives in: threat detection, encryption with managed keys, storage hardening, advanced IAM, and account-wide visibility. Most fit the Free Tier or cost cents – tear everything down when done.

16.1 Threat detection: GuardDuty & Security Hub

THEORY

GuardDuty continuously analyses CloudTrail, VPC Flow Logs, and DNS logs with managed threat intelligence – it surfaces things like crypto-mining instances, credential exfiltration, and contact with known-bad IPs without you writing a single rule. **Security Hub** aggregates GuardDuty plus best-practice benchmark findings (CIS, AWS Foundational) into one prioritised view. Together they are the cloud-native version of your SIEM from Module 8.

```
aws guardduty create-detector --enable
aws securityhub enable-security-hub --enable-default-standards
# turn on VPC Flow Logs so GuardDuty (and you) can see network behaviour
aws ec2 create-flow-logs --resource-type VPC --resource-ids vpc-xxxx \
  --traffic-type ALL --log-destination-type cloud-watch-logs ...
```

RED TEAM – ATTACK

GuardDuty ships a safe way to prove it works – generate sample findings, then watch them appear:

```
aws guardduty create-sample-findings --detector-id <id> \
  --finding-types "UnauthorizedAccess:EC2/SSHBruteForce"
aws guardduty list-findings --detector-id <id>
```

This is the cloud echo of the SSH brute force you ran in Module 6 – same attack, detected by a managed service.

16.2 Encryption with KMS

THEORY

KMS (Key Management Service) manages encryption keys and controls who can use them via key policies. The pattern is "encrypt everything at rest" – EBS volumes, S3 objects, RDS, secrets – with KMS keys whose *use* is itself an IAM-controlled, audited action. A stolen disk or bucket is worthless without the key, and every decryption is logged in CloudTrail.

```
aws kms create-key --description "lab data key"
aws kms create-alias --alias-name alias/lab --target-key-id <key-id>
# now reference the key when creating encrypted resources (S3, EBS, RDS, Secrets
  Manager)
```

16.3 S3 hardening

WATCH OUT

Public S3 buckets are one of the most common real-world data breaches. Defaults to enforce on every bucket: **Block Public Access** on, default encryption with your KMS key, versioning on, and access logging enabled. Grant access by least-privilege bucket/IAM policy – never by making the bucket public.

```
aws s3api put-public-access-block --bucket lab-data \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,
    RestrictPublicBuckets=true
aws s3api put-bucket-encryption --bucket lab-data \
  --server-side-encryption-configuration '{"Rules":[{"
    ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"aws:kms"}}]}'
```

16.4 Secrets Manager

```
# the AWS-native equivalent of the Vault pattern from Module~\ref{mod:iac}
aws secretsmanager create-secret --name lab/db --secret-string '{"password":"x"}'
aws secretsmanager get-secret-value --secret-id lab/db # apps fetch at runtime
# enable automatic rotation so the secret changes itself on a schedule
```

16.5 IAM at depth

THEORY

Beyond basic least-privilege policies, real IAM uses: **roles with AssumeRole** (temporary credentials instead of long-lived keys), **permission boundaries** (a ceiling on what a role can ever grant), **Service Control Policies** (org-wide guardrails no account can exceed), and **IAM Access Analyzer** (which flags resources shared outside your account and can *generate* a least-privilege policy from real usage). These are how you scale least privilege across many accounts.

```
# let Access Analyzer find risky external access automatically
aws accessanalyzer create-analyzer --analyzer-name lab --type ACCOUNT
aws accessanalyzer list-findings --analyzer-arn <arn>
# assume a scoped role for a task instead of using static keys
aws sts assume-role --role-arn arn:aws:iam::<acct>:role/deploy --role-session-name ci
```

VERIFY IT WORKS

GuardDuty shows your sample finding and a real one once you generate suspicious activity; an EBS volume and S3 bucket are encrypted with your KMS key (visible in their properties); Block Public Access is on for every bucket; and Access Analyzer reports no unintended external sharing. That set of checks is essentially a cloud security baseline audit – run it like one.

WATCH OUT

Cost discipline: GuardDuty, Security Hub, flow logs, and KMS all bill. Disable detectors and delete sample resources when you finish a session, and set a billing alarm. The same teardown habit from Module 9 applies doubly here.

MODULE 17

Monitoring & Observability

Your SIEM (Module 8) handles *logs*. A complete picture needs the other two pillars – *metrics* and *traces* – and the alerting pipeline that turns them into action. Security and reliability overlap heavily here: the same anomaly that signals an outage often signals an attack, and modern security engineers are expected to work the observability stack, not just the SIEM.

THEORY

The three pillars. **Logs** are discrete events ("user X failed login"). **Metrics** are numbers over time ("failed-logins-per-minute = 240"). **Traces** follow one request across many services ("this API call touched auth, db, and cache"). Logs tell you *what happened*; metrics tell you *how much and how fast* (great for spotting spikes and trends); traces tell you *where* in a distributed system. A brute-force attack is a few hundred log lines – or one unmistakable spike on a metric, alerting you in seconds.

17.1 Stand up the metrics stack

🎯 OBJECTIVE

Prometheus collecting metrics from your hosts, Grafana dashboarding them, and Alertmanager firing on a security-relevant threshold.

```
# node_exporter on each host exposes system metrics; Prometheus scrapes them
docker run -d -p 9100:9100 prom/node-exporter
docker run -d -p 9090:9090 -v ./prometheus.yml:/etc/prometheus/prometheus.yml prom/
    prometheus
docker run -d -p 3000:3000 grafana/grafana # dashboards at :3000
```

```
# prometheus.yml -- scrape the fleet, load alert rules
scrape_configs:
  - job_name: nodes
    static_configs:
      - targets: ['10.10.10.20:9100', '10.10.30.10:9100']
rule_files: ['alerts.yml']
alerting:
  alertmanagers: [{ static_configs: [{ targets: ['alertmanager:9093'] }] }]
```

17.2 Alert on security signals, not just outages

THEORY

The same pipeline that pages an SRE about high CPU can page *you* about an attack. Write alert rules on security-meaningful metrics: a surge in authentication failures, an unusual spike in outbound bytes (possible exfiltration), a sudden jump in 4xx/5xx from one client (scanning), or a process-count explosion (fork bomb / crypto-miner).


```
# alerts.yml -- a brute-force / exfil style rule expressed as a metric
groups:
- name: security
  rules:
  - alert: AuthFailureSpike
    expr: rate(node_failed_logins_total[2m]) > 1
    for: 1m
    labels: { severity: critical }
    annotations: { summary: "Auth-failure spike on {{ $labels.instance }}" }
  - alert: EgressVolumeSpike
    expr: rate(node_network_transmit_bytes_total[5m]) > 5e7
    annotations: { summary: "Unusual outbound volume on {{ $labels.instance }}" }
```

VERIFY IT WORKS

Re-run the SSH brute force from Module 6. In Grafana the failed-login metric spikes visibly within seconds, and Alertmanager fires **AuthFailureSpike** – the *same* attack your SIEM rule caught in logs, now caught faster as a metric. Two independent detections of one event: defence in depth applied to telemetry.

17.3 Traces and the observability/SIEM divide

THEORY

OpenTelemetry is the vendor-neutral standard for emitting traces (and metrics/logs) from applications; a backend like Tempo/Jaeger stores them. Security relevance: traces expose a request's full path, so you can spot a call reaching a service it never should, or an injection propagating through a chain. **Where the line sits:** observability stacks optimise for real-time operational signal and short retention; the SIEM optimises for security correlation, long retention, and investigation. Mature shops feed key security metrics/traces *into* the SIEM and keep both. Knowing when to use which is a senior instinct worth building now.

WATCH OUT

Don't drown in dashboards. A wall of green graphs nobody reads is theatre. Every panel should answer a question, and every alert must be *actionable* – if it fires and the responder shrugs, it is noise that trains people to ignore the next one. Tune alert thresholds exactly as you tuned IDS/WAF rules: start loose, measure false positives, tighten.

Enterprise relevance: the SRE/security overlap is one of the fastest-growing parts of the field. "I built a Prometheus/Grafana/Alertmanager pipeline with security alerting and understand how it complements a SIEM" signals you can operate in a modern platform team, not just a classic SOC.

MODULE 18

Detection Engineering at Depth

Module 8 wrote a first custom rule. Detection engineering as a *discipline* is bigger: a repeatable lifecycle, measured coverage against a threat framework, validation that your detections actually fire, and proactive hunting for what slips past them. This module turns your SIEM from "collects logs" into "deliberately catches attacker behaviour."

18.1 The detection lifecycle

THEORY

A detection is built, not found. The loop: (1) pick a *threat* (a real ATT&CK technique relevant to your environment); (2) understand its *telemetry* – what log/event it produces; (3) write the *detection* (decoder + rule, or Sigma); (4) *validate* it by safely emulating the attack; (5) *tune* out false positives; (6) document and measure coverage. Repeat. The difference from monitoring is intent: you decide what to catch, then prove you catch it.

18.2 Map coverage to MITRE ATT&CK

THEORY

MITRE ATT&CK is the shared catalogue of attacker techniques. Mapping each detection to a technique ID turns a pile of rules into a measurable *coverage map*: you can see, visually, which tactics you detect and where you are blind. Every attack you ran in this lab maps to a technique – here is the lab's own attack-to-detection matrix.

Attack (lab mod- ule)	ATT&CK	Telemetry	Detection / control
Port scan (M.6)	T1046	firewall drops, flows	Suricata scan sig; flow anomaly
SSH brute force (M.6)	T1110	auth.log failures	Wazuh freq rule + Fail2Ban ban
Web scan / dir brute (M.6)	T1595	nginx 404 flood	ModSecurity/CRS; rate rule
SQLi / XSS (M.6)	T1190	WAF audit log	ModSecurity CRS block
Directory enum (M.1)	T1087	LDAP/AD logs	directory query alert
Kerberoasting (M.1)	T1558.003	TGS-REQ events	service-ticket request alert
Container (M.13)	T1059	syscall (Falco)	Falco "shell in container"
Cloud cred abuse (M.16)	T1078	CloudTrail	GuardDuty finding

🕒 OBJECTIVE

A coverage map of your lab's detections rendered on the ATT&CK matrix, with at least one validated detection per tactic you care about.

```
# the ATT&CK Navigator renders a colour-coded coverage layer from a JSON file
# tag every Wazuh/Sigma rule with its technique id, export, and visualise:
#   green = detected, yellow = partial, red = blind spot
```

18.3 Validate detections by emulating attacks

THEORY

An untested detection is a hope, not a control. **Atomic Red Team** provides small, safe, scripted executions of individual ATT&CK techniques so you can fire a technique on demand and confirm your detection lights up – "detection-as-code", tested like software. If the atomic runs and nothing alerts, you have found a blind spot before the attacker does.

```
# run a single technique safely and check your SIEM reacts (lab hosts only)
Invoke-AtomicTest T1110.001          # password guessing -> expect your brute-force
    rule
Invoke-AtomicTest T1059.004          # unix shell          -> expect Falco/exec alert
# then confirm in the dashboard that the matching detection fired
```

18.4 Threat hunting: look for what rules miss

THEORY

Detections catch *known* patterns; **threat hunting** proactively searches the data for the unknown, starting from a hypothesis ("if an attacker were living off the land, I'd expect unusual parent-child process trees / rare outbound destinations / service accounts logging in interactively"). Hunting findings become tomorrow's detections – it is the feedback loop that grows coverage.

```
# example hunt queries against your aggregated logs
# - beaconing: same src -> same dst at suspiciously regular intervals
# - rare process: binaries seen on only one host, once
# - off-hours admin: privileged logins outside business hours
```

VERIFY IT WORKS

You can point at the ATT&CK matrix and say which techniques you detect and which you do not; each claimed detection has a passing Atomic Red Team test behind it; and at least one hunt has produced a new detection. That is detection engineering, not dashboard-watching.

Portability: keep authoring detections as **Sigma** (Module 8) and storing them in git with their ATT&CK tags and test cases. Detections reviewed in pull requests and validated in CI is the same "security as code" idea as Module 15, applied to defence.

MODULE 19

Incident Response & Digital Forensics

Detection tells you something happened; *incident response* is what you do next, under pressure, without making it worse. This is the most glaring gap in most home labs and a core expectation of any security engineering role: when an alert from your SIEM (Module 8), Falco (Module 13), or GuardDuty (Module 16) fires for real, you need a repeatable process.

THEORY

The IR lifecycle (NIST / PICERL). Six phases you run in order: **Preparation** (tooling, runbooks, access ready *before* the incident), **Identification** (confirm it is real, scope it), **Containment** (stop the bleeding without destroying evidence), **Eradication** (remove the foothold), **Recovery** (restore service, verify clean), and **Lessons Learned** (the post-incident review that prevents a repeat). Amateurs jump straight to "wipe and reinstall"; professionals contain, preserve evidence, understand the root cause, *then* remediate.

19.1 Triage: is this real, and how bad?

🎯 OBJECTIVE

Given a fired alert, quickly determine scope, severity, and whether to escalate – before touching the host in a way that destroys evidence.

```
# triage a suspect Linux host (volatile data FIRST, least to most disruptive)
who; last -20                                # who is / was logged in
ps auxf                                       # process tree -- spot odd parents/children
ss -tunap                                     # active connections -- find C2 / exfil
ls -alt /tmp /dev/shm /var/tmp                # common dropper locations, newest first
crontab -l; ls -la /etc/cron.*                # persistence
```

WATCH OUT

Order of volatility. Capture the most fleeting evidence first: RAM and live connections vanish on reboot; disk persists. The instinctive "turn it off and on again" destroys exactly the evidence (running malware, network sockets, decryption keys in memory) that explains the incident. **Isolate, do not power off.**

19.2 Containment without destroying evidence

THEORY

Containment stops spread while preserving the crime scene. Network isolation – pulling the host onto a quarantine VLAN or applying a deny-all firewall rule (your Module 3 skills) – cuts the attacker off without killing the running state you need to analyse. For a compromised cloud instance you isolate via security group; for a pod, a deny-all NetworkPolicy (Module 13).

```
# isolate at the network edge instead of shutting the box down
sudo nft add rule inet filter forward ip saddr 10.10.30.10 drop # quarantine the
host
# OR move it to a quarantine VLAN with no route anywhere but the analyst box
```

19.3 Evidence acquisition & integrity

THEORY

Evidence is only useful if you can prove it was not altered. Acquire RAM and disk images, then hash them immediately – the hash is the integrity seal, and the record of who held the evidence when is the **chain of custody**. Sloppy handling makes findings worthless (and, in a real org, legally inadmissible).

```
# memory acquisition (volatile -- do this before disk)
sudo apt install avml -y && sudo avml /evidence/mem.lime
# disk image with a forensic copier, then SEAL it with a hash
sudo dc3dd if=/dev/sdb of=/evidence/disk.img hash=sha256 log=/evidence/disk.log
sha256sum /evidence/*.img > /evidence/hashes.txt # integrity seal
# record: who acquired, when, from which host, hash -> chain of custody
```

VERIFY IT WORKS

Re-hashing the image later produces the *same* SHA-256 in `hashes.txt` – proving the evidence is unaltered. That reproducible hash is the difference between "I think the attacker did X" and defensible evidence.

19.4 Analysis: build the timeline

THEORY

The output of an investigation is a **timeline**: what happened, in order, with evidence for each step. Filesystem timestamps, log entries, shell history, and process data combine into a single narrative – initial access → execution → persistence → lateral movement → objective, mapped to the ATT&CK techniques from Module 18.

```
# super-timeline from disk artefacts (Plaso) -- merges many sources by time
log2timeline.py timeline.plaso /evidence/disk.img
psort.py -o l2tcsv -w timeline.csv timeline.plaso
# pull the indicators of compromise (IOCs) you will block/hunt for elsewhere
# malicious IPs, file hashes, domains, attacker usernames
```

19.5 Eradication, recovery, and the post-incident review

THEORY

Eradicate the root cause, not just the symptom – if they got in via an unpatched CVE (Module 7) or a leaked key (Module 14), removing the malware without closing that door means they return tonight. **Recover** from known-good backups and *verify* the system is clean before returning it to service. Then the most valuable phase: the blameless **post-incident review** – what happened, what detection caught it (or should have), and which control or detection you will add so it cannot recur.

Close the loop with the rest of the lab. Every IR turns into improvements: a new SIEM/Sigma detection (Module 8, 18), a firewall or NetworkPolicy change (Module 3, 13), a patch or secret rotation (Module 7, 14), and an IOC list to hunt across the fleet. The incident makes the whole environment stronger – that feedback loop *is* security engineering.

RED TEAM – ATTACK

Run a full IR drill. Have a teammate (or your own scripted attack) compromise the DMZ web server, then work the lifecycle end to end: triage → contain (isolate, don't kill) → acquire + hash → build the timeline → identify root cause → eradicate + recover → write the review. This drill, documented, is one of the strongest portfolio pieces a junior candidate can show – it proves you can handle a real incident, not just configure tools.

MODULE 20

Capstone: Purple-Team Exercise

Individual modules prove individual skills. This capstone proves you can think across the whole stack: model the threats to your lab, run a realistic multi-stage attack end to end (red), and verify that each stage is prevented or detected by the controls you built (blue). Doing both sides is what "purple team" means – and it is the single best portfolio piece you can show.

20.1 Threat-model the lab first

THEORY

Before attacking, model what you are defending. A lightweight threat model asks four questions: *What are we building?* (your architecture diagram from Module 0), *What can go wrong?* (use STRIDE – Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), *What are we doing about it?* (the controls in each module), and *Did we do enough?* (the validation below). The model tells you which attack path matters most – here, the public DMZ web server pivoting toward the directory.

OBJECTIVE

A written threat model identifying the highest-risk path through the lab, and the specific controls expected to break that path at each step.

20.2 The kill chain: attack the whole lab

RED TEAM – ATTACK

Run this staged attack from `attacker-kali` against *your own* lab. At each stage, note which control should fire – then check it did.

1. **Recon** – `nmap` the exposed DMZ. *Expect:* firewall drops + Suricata scan alert (T1046).
2. **Initial access** – web attack against the DMZ app. *Expect:* ModSecurity/CRS blocks SQLi/XSS at the proxy (T1190).
3. **Execution/foothold** – land in a container on the web host. *Expect:* hardened image (non-root, cap-drop) limits you; Falco alerts on the shell (T1059).
4. **Lateral movement** – from the DMZ, try to reach the trusted LAN. *Expect:* the DMZ→LAN deny (Module 3) and K8s NetworkPolicy stop you cold.
5. **Privilege/identity** – enumerate and Kerberoast the directory. *Expect:* directory alerts (T1087, T1558.003); least-privilege RBAC limits the prize.
6. **Persistence/exfil** – attempt outbound C2 / data theft. *Expect:* egress firewall rules + proxy logging + (cloud) GuardDuty catch it.

20.3 Verify the defence, end to end

VERIFY IT WORKS

For every stage above, confirm in the dashboard that the expected control fired – and wherever one did *not*, you have found a real gap to fix. Success is not "the attack failed everywhere"; it is "every stage was either prevented or detected, and I can show the evidence trail." Write it up as an incident timeline: stage, technique, control, evidence. That document is exactly what a security engineer produces after a purple-team exercise.

THEORY

Why this is the real test. Any single control can be bypassed; defence-in-depth means an attacker must defeat *every* layer without tripping *any* detection. By running the whole chain you stop thinking in isolated tools and start thinking in *attack paths and layered controls* – which is the actual mindset of the job.

20.4 Where to take it next

- **Re-run after every change.** Treat the capstone as a regression test for your security posture: change a control, re-run the chain, confirm nothing regressed.
- **Grow the matrix.** Add techniques to the ATT&CK coverage map (Module 18) and build the detection that catches each new one.
- **Automate it.** Script the chain and the verification so the whole purple-team run is one command – continuous validation, the same idea as CI gates (Module 15) applied to security itself.
- **Document publicly.** A written-up purple-team exercise on this lab – architecture, attack paths, controls, evidence – is a stronger portfolio signal than any certificate alone.

You have now built, attacked, and defended a segmented network, a directory, proxies with inspection, a VPN, host and container hardening, a Kubernetes cluster, infrastructure-as-code with scanning, a secure CI/CD supply chain, a cloud environment, and a detection pipeline measured against ATT&CK – then proved it holds together under a full kill chain. That is security engineering.

MODULE 21

Skills Checklist & Study Path

Use this as your progress tracker. Tick a row only when you have done it *and* the Verify box passed – that is the difference between "watched a tutorial" and "can do it".

21.1 Mastery checklist

✓	Skill	Module
☐	Stand up a directory (AD/FreeIPA); join hosts	Identity
☐	Disable local-only auth; keep break-glass access	Identity
☐	Grant access by group (RBAC), not per host	Identity
☐	Enumerate a directory & explain Kerberoasting	Identity
☐	Diagnose a network fault layer by layer	Foundations
☐	Subnet a network and explain /24 vs /28	Foundations
☐	Read a routing table and trace a packet	Foundations
☐	Explain SNAT vs DNAT from memory	Found./Firewall
☐	Bring up WAN + LAN with DHCP on the firewall	Firewall
☐	Write a default-deny stateful rule set	Firewall
☐	Block ports / allow only specific source IPs	Firewall
☐	Configure NAT and a port forward	Firewall
☐	Block SSH from the internet	Firewall
☐	Build DMZ segmentation (deny DMZ→LAN)	Firewall
☐	Run a forward proxy with domain filtering	Proxies
☐	Block YouTube and log every request	Proxies
☐	Force traffic through a transparent proxy	Proxies
☐	Intercept HTTPS with SSL Bump (bump-and-splice)	Proxies
☐	Authenticate the proxy against the directory	Proxies
☐	Front a server with a reverse proxy	Proxies
☐	Stand up a WireGuard VPN into the lab	VPN
☐	Explain WireGuard vs OpenVPN trade-offs	VPN
☐	Run nmap / hydra / nikto against the lab	IDS/IPS
☐	Get Suricata alerts, then promote to IPS drop	IDS/IPS
☐	Block web attacks with ModSecurity + CRS	IDS/IPS
☐	Harden SSH (keys only, no root)	Hardening
☐	Configure ufw and Fail2Ban	Hardening
☐	Add MFA (TOTP) to SSH and VPN	Hardening
☐	Enforce config across the fleet with Ansible	Hardening

✓	Skill	Module
☐	Run vuln scans; close the find→fix→verify loop	Hardening
☐	Centralise logs into a dashboard (SIEM)	Integrated
☐	Write a custom Wazuh detection rule (+ MITRE)	Integrated
☐	Trace an attack across firewall + IDS + dashboard	Integrated
☐	Build a VPC with public/private subnets	Cloud
☐	Configure Security Groups, NACLs, NAT GW	Cloud
☐	Stand up a bastion, ALB, and WAF	Cloud
☐	Write a least-privilege IAM policy	Cloud
☐	Audit activity with CloudTrail/CloudWatch	Cloud
☐	Run a two-tier internal CA; issue & revoke certs	PKI
☐	Configure and test mutual TLS (mTLS)	PKI
☐	Write nftables policy with sets	Linux/nft
☐	Use namespaces, capabilities, seccomp, AppArmor	Linux/nft
☐	Build a non-root, scanned, hardened container	Containers
☐	Harden K8s: RBAC, NetworkPolicy, PSS, admission	Containers
☐	Detect runtime container threats with Falco	Containers
☐	Provision with Terraform; scan IaC (trivy/checkov)	IaC
☐	Manage secrets with Vault (dynamic creds)	IaC
☐	Build a gated CI pipeline (secret/SCA/SAST/IaC)	CI/CD
☐	Produce an SBOM and sign artifacts (cosign)	CI/CD
☐	Use GuardDuty, KMS, S3 hardening, IAM at depth	AWS depth
☐	Map detections to ATT&CK; validate with atomics	Det. Eng.
☐	Threat-hunt and turn hunts into detections	Det. Eng.
☐	Run a full purple-team kill chain end to end	Capstone
☐	Deploy Sysmon; read key Windows Event IDs	Windows
☐	Detect Kerberoasting/DCSync/PtH on Windows	Windows
☐	Harden PAM, systemd units, audit journald	Linux/nft
☐	Build Prometheus/Grafana/Alertmanager pipeline	Observability
☐	Prioritise vulns with CVSS + EPSS + KEV	Hardening
☐	Run the IR lifecycle: triage→contain→eradicate	Incident Resp.
☐	Acquire & hash evidence; build a timeline	Incident Resp.

21.2 Suggested order & pacing

Do the modules in order – each builds on the last. A realistic, sustainable pace:

Week	Focus	Outcome
1	Blueprint + Identity	Lab VMs built; directory live, hosts joined

Week	Focus	Outcome
2	Foundations + Firewall	Networking solid; NAT + full rule set working
3	Proxies	Filtering, SSL Bump, reverse proxy + directory auth
4	VPN + IDS/IPS	Tunnel in; attacks detected; IPS + WAF blocking
5	Hardening	Hosts hardened; MFA, Ansible & vuln-scan loop
6	Integrated detection	Dashboard live; custom rules; chains traced
7–8	Cloud	Architecture rebuilt in AWS; tear-down disciplined
<i>Advanced track – another 6–8 weeks, in order:</i>		
9	PKI + Linux internals/nftables	Internal CA, mTLS; kernel security, PAM/systemd/journald
10	Windows security & detection	Sysmon + Event IDs; detect the AD attacks
11–12	Containers + Kubernetes	Hardened images; locked-down cluster + Falco
13	IaC + secrets (Terraform/-Vault)	Infra as code, scanned; no plaintext secrets
14	CI/CD + supply chain	Gated pipeline; SBOM + signed artifacts
15	Advanced AWS + observability	GuardDuty/KMS; Prometheus/-Grafana alerting
16	Detection depth + incident response	ATT&CK coverage map; full IR drill
17	Capstone purple team	Full kill chain run & defended, written up

Where this points next. The skills here map directly onto CompTIA Network+ and Security+, and the offensive practice prepares you for platforms like TryHackMe and Hack The Box. If you want a career signal, those two certifications plus this lab as a portfolio project are a strong combination.

MODULE A

Command Cheat-Sheet

A.1 Networking & diagnostics

```
ip addr / ip route / ip link      # addresses, routes, interfaces
ss -tulnp                         # listening TCP/UDP ports + process
ping / traceroute / mtr          # reachability and path
dig <name> +short / +trace        # DNS resolution
tcpdump -ni any 'tcp port 80'    # live packet capture
nc -vz <host> <port>             # is a port open?
```

A.2 Firewall (iptables / ufw)

```
iptables -L -n -v --line-numbers  # list filter rules with counters
iptables -t nat -L -n -v         # list NAT rules
iptables -I FORWARD <n> ...      # insert at position n (order matters!)
netfilter-persistent save        # persist rules across reboot
ufw status verbose               # ufw policy summary
```

A.3 Services & logs

```
systemctl status|restart|enable <svc>
journalctl -u <svc> -f           # follow a service's logs
tail -f /var/log/auth.log        # live SSH auth attempts
tail -f /var/log/suricata/eve.json # live IDS alerts
fail2ban-client status sshd      # who is banned
```

MODULE B

Troubleshooting Playbook

Symptom	First things to check
No internet from LAN	<code>ip_forward=1</code> ? MASQUERADE rule present? default route on client points at firewall?
Port forward not working	DNAT in PREROUTING <i>and</i> a matching FORWARD accept? backend service actually listening?
DMZ can still reach LAN	the DMZ→LAN DROP rule is below an earlier ACCEPT – check rule order.
Locked out of SSH	you disabled passwords before the key worked – use VM console to fix <code>sshd_config</code> .
Proxy not filtering	client bypassing it (explicit proxy unset) – use transparent interception.
No IDS alerts	Suricata watching the right interface? <code>local.rules</code> loaded? traffic actually crossing the sensor?

MODULE C

Where to Keep Learning

- **Hands-on platforms:** TryHackMe (guided, beginner-friendly paths for network security and blue team), Hack The Box (less guided, more realistic), OverTheWire (free Linux + security fundamentals).
- **Foundational theory:** Professor Messer's free Network+ and Security+ video courses; the book *Computer Networking: A Top-Down Approach*.
- **Tooling docs worth reading end to end:** the pfSense/OPNsense handbook, the Suricata user guide, and the WireGuard quick-start.
- **Certifications that match this lab:** CompTIA Network+ → Security+ → (optionally) Cisco CCNA for deeper networking or a blue-team cert for detection.
- **Habit:** keep this lab alive. Break something on purpose each week, then fix it. The attack-/defend loop is the whole game.

You now hold a complete, buildable security network. Work through it module by module, tick every box, and you will have done – not just read – security engineering.